

MIPT & YSDA · AUTUMN 2026

# Deep Generative Models

## Lecture 4

Roman Isachenko

Moscow Institute of Physics and Technology  
Yandex School of Data Analysis

# Recap of Previous Lecture

## MLE for LVM

$$\sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i) = \sum_{i=1}^n \log \int p_{\theta}(\mathbf{x}_i|\mathbf{z}_i)p(\mathbf{z}_i)d\mathbf{z}_i \rightarrow \max_{\theta}.$$

## A Naive Lower Bound

$$\begin{aligned} \log p_{\theta}(\mathbf{x}) &= \log \mathbb{E}_{p(\mathbf{z})} p_{\theta}(\mathbf{x}|\mathbf{z}) \geq \mathbb{E}_{p(\mathbf{z})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) \\ &\approx \frac{1}{K} \sum_{k=1}^K \log p_{\theta}(\mathbf{x}|\mathbf{z}_k), \quad \text{where } \mathbf{z}_k \sim p(\mathbf{z}). \end{aligned}$$

## Variational Evidence Lower Bound (ELBO)

$$\begin{aligned} \mathcal{L}_{q,\theta}(\mathbf{x}) &= \mathbb{E}_q \log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} \\ \log p_{\theta}(\mathbf{x}) &= \mathcal{L}_{q,\theta}(\mathbf{x}) + D_{\text{KL}}(q(\mathbf{z})\|p_{\theta}(\mathbf{z}|\mathbf{x})) \geq \mathcal{L}_{q,\theta}(\mathbf{x}). \end{aligned}$$

# Recap of Previous Lecture

## Variational Evidence Lower Bound (ELBO)

$$\mathcal{L}_{q,\theta}(\mathbf{x}) = \mathbb{E}_q \log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} \leq \log p_{\theta}(\mathbf{x})$$

$$\mathcal{L}_{q,\theta}(\mathbf{x}) = \int q(\mathbf{z}) \log \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} d\mathbf{z} = \mathbb{E}_q \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q(\mathbf{z})\|p(\mathbf{z}))$$

## Log-Likelihood Decomposition

$$\log p_{\theta}(\mathbf{x}) = \mathbb{E}_q \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q(\mathbf{z})\|p(\mathbf{z})) + D_{\text{KL}}(q(\mathbf{z})\|p_{\theta}(\mathbf{z}|\mathbf{x})).$$

- Maximize the ELBO as a surrogate for log-likelihood:

$$\max_{\theta} \log p_{\theta}(\mathbf{x}) \longrightarrow \max_{q,\theta} \mathcal{L}_{q,\theta}(\mathbf{x})$$

- Maximizing the ELBO with respect to the variational distribution  $q$  is equivalent to minimizing the KL divergence:

$$\arg \max_q \mathcal{L}_{q,\theta}(\mathbf{x}) \equiv \arg \min_q D_{\text{KL}}(q(\mathbf{z})\|p_{\theta}(\mathbf{z}|\mathbf{x})).$$

# Recap of Previous Lecture

$$\begin{aligned}\mathcal{L}_{q,\theta}(\mathbf{x}) &= \mathbb{E}_q \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q(\mathbf{z})\|p(\mathbf{z})) \\ &= \mathbb{E}_q \left[ \log p_{\theta}(\mathbf{x}|\mathbf{z}) - \log \frac{q(\mathbf{z})}{p(\mathbf{z})} \right] \rightarrow \max_{q,\theta}.\end{aligned}$$

## Variational Posterior

$$q^*(\mathbf{z}) = \arg \max_q \mathcal{L}_{q,\theta^*}(\mathbf{x}) = \arg \min_q D_{\text{KL}}(q(\mathbf{z})\|p_{\theta^*}(\mathbf{z}|\mathbf{x})) = p_{\theta^*}(\mathbf{z}|\mathbf{x}).$$

## Amortized Variational Inference

We restrict the family of possible distributions  $q(\mathbf{z})$  to a parametric class  $q_{\phi}(\mathbf{z}|\mathbf{x})$ , conditioned on data  $\mathbf{x}$  and parameterized by  $\phi$ .

## Gradient Update

$$\begin{bmatrix} \phi_k \\ \theta_k \end{bmatrix} = \begin{bmatrix} \phi_{k-1} + \eta \cdot \nabla_{\phi} \mathcal{L}_{\phi,\theta}(\mathbf{x}) \\ \theta_{k-1} + \eta \cdot \nabla_{\theta} \mathcal{L}_{\phi,\theta}(\mathbf{x}) \end{bmatrix} \Big|_{(\phi_{k-1}, \theta_{k-1})}$$

# Recap of Previous Lecture

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_q \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \rightarrow \max_{\phi, \theta}.$$

## Gradient $\nabla_{\theta} \mathcal{L}_{\phi, \theta}(\mathbf{x})$

$$\begin{aligned} \nabla_{\theta} \mathcal{L}_{\phi, \theta}(\mathbf{x}) &= \int q_{\phi}(\mathbf{z}|\mathbf{x}) \nabla_{\theta} \log p_{\theta}(\mathbf{x}|\mathbf{z}) d\mathbf{z} \\ &\approx \nabla_{\theta} \log p_{\theta}(\mathbf{x}|\mathbf{z}^*), \quad \mathbf{z}^* \sim q_{\phi}(\mathbf{z}|\mathbf{x}). \end{aligned}$$

## Gradient $\nabla_{\phi} \mathcal{L}_{\phi, \theta}(\mathbf{x})$

$$\begin{aligned} \nabla_{\phi} \mathcal{L}_{\phi, \theta}(\mathbf{x}) &= \int p(\epsilon) \nabla_{\phi} \log p_{\theta}(\mathbf{x}|\mathbf{g}_{\phi}(\mathbf{x}, \epsilon)) d\epsilon - \nabla_{\phi} D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \\ &\approx \nabla_{\phi} \log p_{\theta}(\mathbf{x}|\mathbf{g}_{\phi}(\mathbf{x}, \epsilon^*)) - \nabla_{\phi} D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \end{aligned}$$

## Reparametrization: A Gaussian Example

$$\begin{aligned} p(\epsilon) &= \mathcal{N}(0, \mathbf{I}); \quad q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathbf{x}), \boldsymbol{\sigma}_{\phi}^2(\mathbf{x})). \\ \mathbf{z} &= \mathbf{g}_{\phi}(\mathbf{x}, \epsilon) = \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \odot \epsilon + \boldsymbol{\mu}_{\phi}(\mathbf{x}). \end{aligned}$$

# Recap of Previous Lecture

## Training

1. Sample  $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ ,  $\epsilon \sim p(\epsilon)$ .
2. Reparametrize  $\mathbf{z} = \mathbf{g}_{\phi}(\mathbf{x}, \epsilon)$ .
3. Compute the ELBO:

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) \approx \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})).$$

4. Update  $\phi$ ,  $\theta$  via stochastic gradient ascent.

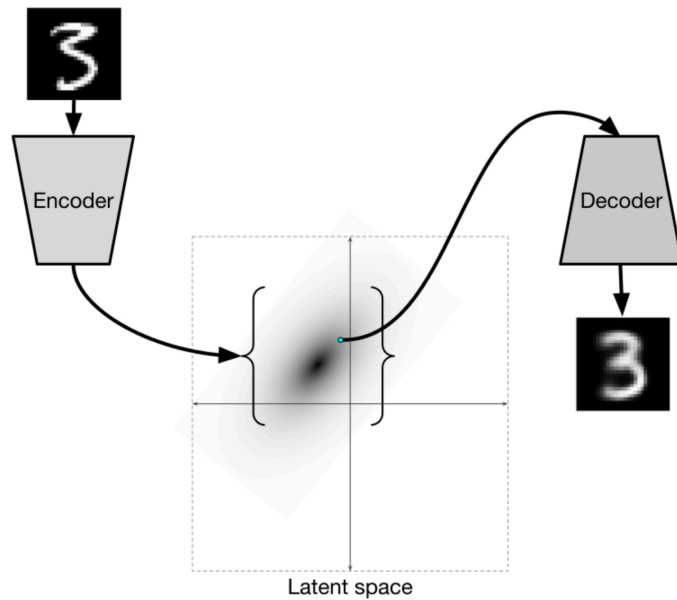
## Sampling

1. Sample  $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ .
2. Sample  $\mathbf{x} \sim p_{\theta}(\mathbf{x}|\mathbf{z})$ .

**Note:** The encoder  $q_{\phi}(\mathbf{z}|\mathbf{x})$  isn't needed during generation.

# Recap of Previous Lecture

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_q \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$$

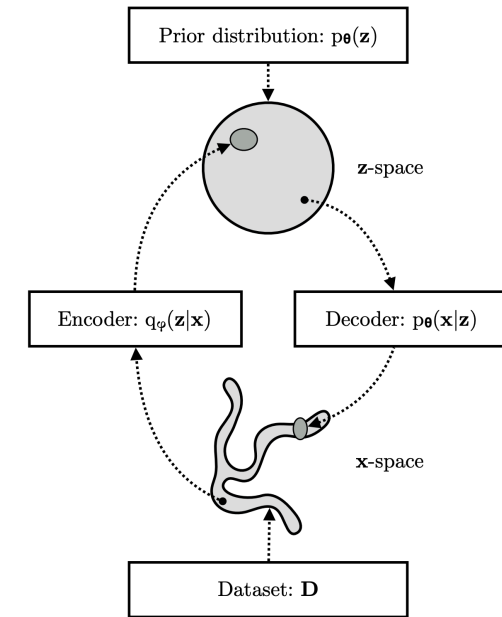


## Encoder $q_{\phi}(\mathbf{z}|\mathbf{x})$

Predicts  $\mu_{\phi}(\mathbf{x})$  and  $\sigma_{\phi}(\mathbf{x})$ .

## Decoder $p_{\theta}(\mathbf{x}|\mathbf{z})$

Predicts parameters of the data distribution.



## Prior $p(\mathbf{z})$

Typically  $\mathcal{N}(0, \mathbf{I})$ ; used for sampling.

# Outline

01 ELBO Surgery and Optimal VAE Prior

02 Discrete Latent VAEs

Discrete VAE Latent Representations  
Vector Quantized VAE (VQ-VAE)

03 Likelihood-Free Learning

04 Generative Adversarial Networks (GAN)

# Outline

## 01 ELBO Surgery and Optimal VAE Prior

## 02 Discrete Latent VAEs

Discrete VAE Latent Representations  
Vector Quantized VAE (VQ-VAE)

## 03 Likelihood-Free Learning

## 04 Generative Adversarial Networks (GAN)

# ELBO Surgery

$$\frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) = \frac{1}{n} \sum_{i=1}^n \left[ \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) \right].$$

# ELBO Surgery

$$\frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) = \frac{1}{n} \sum_{i=1}^n \left[ \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) \right].$$

## Theorem

$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}];$$

# ELBO Surgery

$$\frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) = \frac{1}{n} \sum_{i=1}^n \left[ \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) \right].$$

## Theorem

$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}];$$

### Aggregated Variational Posterior

$$q_{\text{agg}, \phi}(\mathbf{z}) = \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{z}|\mathbf{x}_i)$$

- $\mathbb{I}_q[\mathbf{x}, \mathbf{z}]$  is the mutual information between  $\mathbf{x}$  and  $\mathbf{z}$  under the data distribution  $p_{\text{data}}(\mathbf{x})$  and  $q_{\phi}(\mathbf{z}|\mathbf{x})$ .
- The first term encourages  $q_{\text{agg}, \phi}(\mathbf{z})$  to match the prior  $p(\mathbf{z})$ .
- The second term reduces the information about  $\mathbf{x}$  encoded in  $\mathbf{z}$ .

# ELBO Surgery

$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}].$$

## Proof

$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z})} d\mathbf{z}$$

# ELBO Surgery

$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}].$$

## Proof

$$\begin{aligned} \frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) &= \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z})} d\mathbf{z} \\ &= \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\text{agg},\phi}(\mathbf{z}) q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z}) q_{\text{agg},\phi}(\mathbf{z})} d\mathbf{z} \end{aligned}$$

# ELBO Surgery

$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}].$$

## Proof

$$\begin{aligned} \frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) &= \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z})} d\mathbf{z} \\ &= \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\text{agg},\phi}(\mathbf{z}) q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z}) q_{\text{agg},\phi}(\mathbf{z})} d\mathbf{z} \\ &= \int \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\text{agg},\phi}(\mathbf{z})}{p(\mathbf{z})} d\mathbf{z} + \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{q_{\text{agg},\phi}(\mathbf{z})} d\mathbf{z} \end{aligned}$$

# ELBO Surgery

$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}].$$

## Proof

$$\begin{aligned} \frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) &= \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z})} d\mathbf{z} \\ &= \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\text{agg},\phi}(\mathbf{z}) q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z}) q_{\text{agg},\phi}(\mathbf{z})} d\mathbf{z} \\ &= \int \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\text{agg},\phi}(\mathbf{z})}{p(\mathbf{z})} d\mathbf{z} + \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{q_{\text{agg},\phi}(\mathbf{z})} d\mathbf{z} \\ &= D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z})) + \frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| q_{\text{agg},\phi}(\mathbf{z})) \end{aligned}$$

# ELBO Surgery

$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}].$$

## Proof

$$\begin{aligned} \frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) &= \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z})} d\mathbf{z} \\ &= \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\text{agg},\phi}(\mathbf{z}) q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{p(\mathbf{z}) q_{\text{agg},\phi}(\mathbf{z})} d\mathbf{z} \\ &= \int \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\text{agg},\phi}(\mathbf{z})}{p(\mathbf{z})} d\mathbf{z} + \frac{1}{n} \sum_{i=1}^n \int q_{\phi}(\mathbf{z}|\mathbf{x}_i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}_i)}{q_{\text{agg},\phi}(\mathbf{z})} d\mathbf{z} \\ &= D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z})) + \frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| q_{\text{agg},\phi}(\mathbf{z})) \\ \mathbb{I}_q[\mathbf{x}, \mathbf{z}] &= \frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| q_{\text{agg},\phi}(\mathbf{z})). \end{aligned}$$

# ELBO Surgery

## Revisiting the ELBO

$$\frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) = \frac{1}{n} \sum_{i=1}^n \left[ \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) \right]$$

# ELBO Surgery

## Revisiting the ELBO

$$\begin{aligned}
 \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) &= \frac{1}{n} \sum_{i=1}^n \left[ \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) \right] \\
 &= \underbrace{\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z})}_{\text{Reconstruction Loss}} - \underbrace{\mathbb{I}_q[\mathbf{x}, \mathbf{z}]}_{\text{Mutual Information}} - \underbrace{D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z}))}_{\text{Marginal KL}}
 \end{aligned}$$

# ELBO Surgery

## Revisiting the ELBO

$$\begin{aligned}
 \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) &= \frac{1}{n} \sum_{i=1}^n \left[ \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) \right] \\
 &= \underbrace{\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z})}_{\text{Reconstruction Loss}} - \underbrace{\mathbb{I}_q[\mathbf{x}, \mathbf{z}]}_{\text{Mutual Information}} - \underbrace{D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z}))}_{\text{Marginal KL}}
 \end{aligned}$$

The **prior distribution**  $p(\mathbf{z})$  only appears in the **Marginal KL** term.

# ELBO Surgery

## Revisiting the ELBO

$$\begin{aligned}
 \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) &= \frac{1}{n} \sum_{i=1}^n \left[ \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) \right] \\
 &= \underbrace{\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z})}_{\text{Reconstruction Loss}} - \underbrace{\mathbb{I}_q[\mathbf{x}, \mathbf{z}]}_{\text{Mutual Information}} - \underbrace{D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z}))}_{\text{Marginal KL}}
 \end{aligned}$$

The **prior distribution**  $p(\mathbf{z})$  only appears in the **Marginal KL** term.

## Optimal VAE Prior

$$D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z})) = 0 \quad \Leftrightarrow \quad p(\mathbf{z}) = q_{\text{agg}, \phi}(\mathbf{z}) = \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{z}|\mathbf{x}_i).$$

# ELBO Surgery

## Revisiting the ELBO

$$\begin{aligned}
 \frac{1}{n} \sum_{i=1}^n \mathcal{L}_{\phi, \theta}(\mathbf{x}_i) &= \frac{1}{n} \sum_{i=1}^n [\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z}))] \\
 &= \underbrace{\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}_i)} \log p_{\theta}(\mathbf{x}_i|\mathbf{z})}_{\text{Reconstruction Loss}} - \underbrace{\mathbb{I}_q[\mathbf{x}, \mathbf{z}]}_{\text{Mutual Information}} - \underbrace{D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z}))}_{\text{Marginal KL}}
 \end{aligned}$$

The **prior distribution**  $p(\mathbf{z})$  only appears in the **Marginal KL** term.

## Optimal VAE Prior

$$D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z})) = 0 \quad \Leftrightarrow \quad p(\mathbf{z}) = q_{\text{agg}, \phi}(\mathbf{z}) = \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{z}|\mathbf{x}_i).$$

Hence, the **optimal prior**  $p(\mathbf{z})$  is the **aggregated variational posterior**  $q_{\text{agg}, \phi}(\mathbf{z})$ .

# Marginal KL

$$D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z})\|p(\mathbf{z}))$$

## Training

The decoder receives  $\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x}_i)$ ; over the training set,  $\mathbf{z} \sim q_{\text{agg},\phi}(\mathbf{z})$ .

## Sampling (generation)

The decoder receives  $\mathbf{z} \sim p(\mathbf{z})$ , then generates  $\mathbf{x} \sim p_{\theta}(\mathbf{x}|\mathbf{z})$ .

- $q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathbf{x}), \boldsymbol{\sigma}_{\phi}^2(\mathbf{x}))$  is unimodal.
- A mismatch between the prior and the aggregated posterior can degrade sample quality. Matching these distributions does not by itself eliminate blurriness.

# Marginal KL

$$D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z}))$$

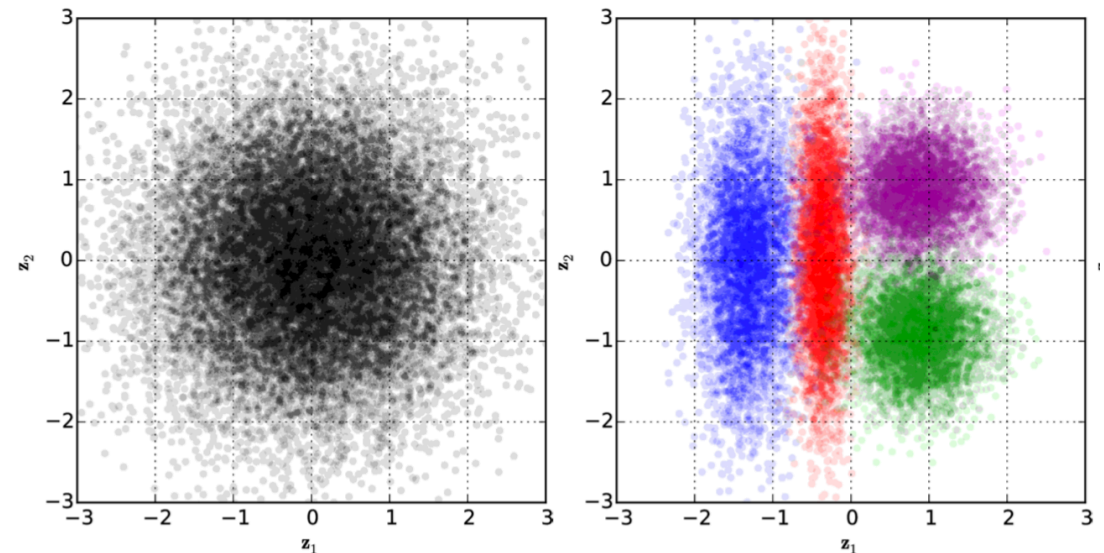
## Training

The decoder receives  $\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x}_i)$ ; over the training set,  $\mathbf{z} \sim q_{\text{agg},\phi}(\mathbf{z})$ .

- $q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathbf{x}), \boldsymbol{\sigma}_{\phi}^2(\mathbf{x}))$  is unimodal.
- A mismatch between the prior and the aggregated posterior can degrade sample quality. Matching these distributions does not by itself eliminate blurriness.

## Sampling (generation)

The decoder receives  $\mathbf{z} \sim p(\mathbf{z})$ , then generates  $\mathbf{x} \sim p_{\theta}(\mathbf{x}|\mathbf{z})$ .



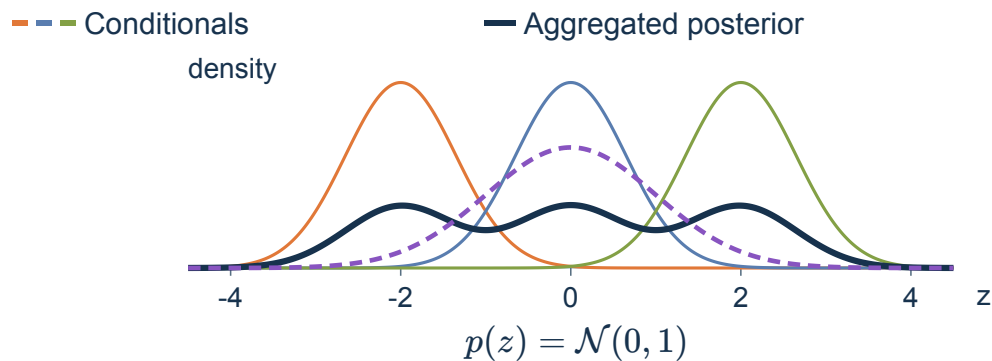
(a) Prior distribution

(b) Posteriors in standard VAE

# Matching the Prior: Which KL Vanishes?

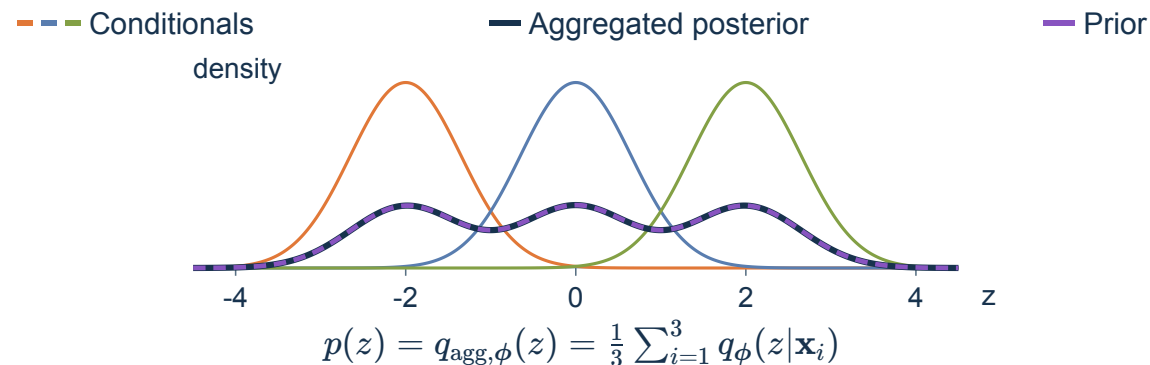
$$\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z})) = D_{\text{KL}}(q_{\text{agg},\phi}(\mathbf{z}) \| p(\mathbf{z})) + \mathbb{I}_q[\mathbf{x}, \mathbf{z}].$$

$$q_{\phi}(z|\mathbf{x}_i) = \mathcal{N}(\mu_i, 0.65^2), \quad (\mu_1, \mu_2, \mu_3) = (-2, 0, 2), \quad p_{\text{data}}(\mathbf{x}_i) = 1/3$$



|                     |       |
|---------------------|-------|
| Marginal KL         | 0.584 |
| Mutual information  | 0.892 |
| Mean conditional KL | 1.475 |

All values in nats; all bars use the same scale.



|                     |       |
|---------------------|-------|
| Marginal KL         | 0.000 |
| Mutual information  | 0.892 |
| Mean conditional KL | 0.892 |

All values in nats; all bars use the same scale.

Matching the prior removes **Marginal KL**. The **mutual information** and encoder remain unchanged.

# ELBO Surgery in Modern Latent Models

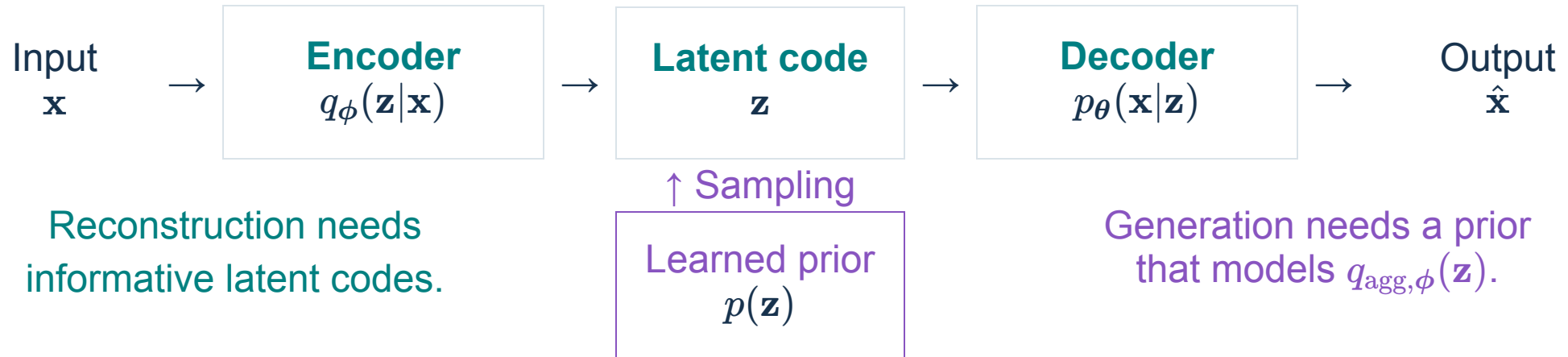
Jointly learn an **autoencoder** and a **generative prior**.

Average conditional KL = **Marginal KL** + **Mutual Information**.

# ELBO Surgery in Modern Latent Models

Jointly learn an **autoencoder** and a **generative prior**.

Average conditional KL = Marginal KL + Mutual Information.

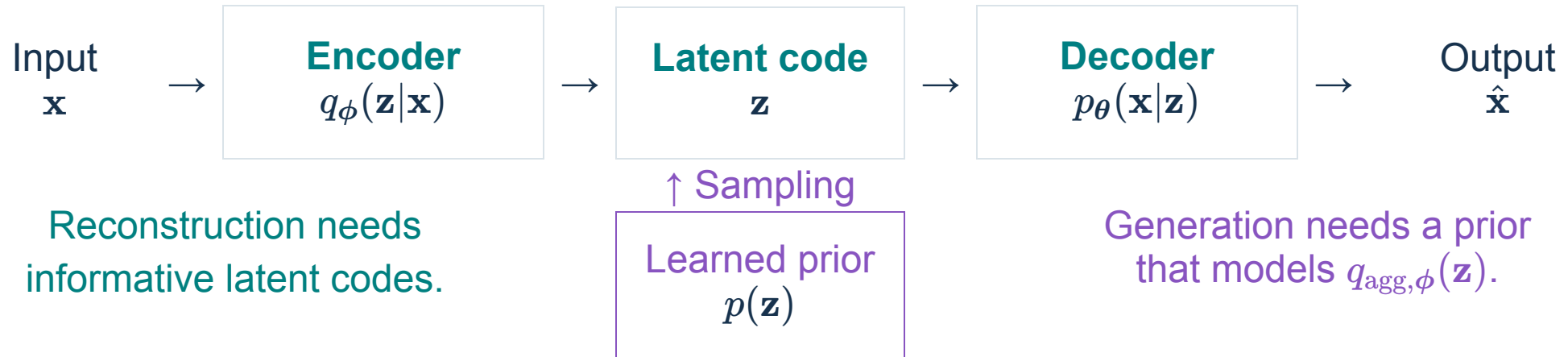


Latents that **reconstruct well** may be **hard for the prior to model**.

# ELBO Surgery in Modern Latent Models

Jointly learn an **autoencoder** and a **generative prior**.

Average conditional KL = Marginal KL + Mutual Information.



Latents that **reconstruct well** may be **hard for the prior to model**.

**GenFirst (2026)** studies this balance when the autoencoder and generative prior are trained jointly.

# From ELBO to Squared Error

Fix the encoder  $q_\phi(\mathbf{z}|\mathbf{x})$ , prior  $p(\mathbf{z})$  and decoder variance  $\sigma^2 > 0$ ; optimize only  $\theta$ .

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{p_{\text{data}}(\mathbf{x}) q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) - \underbrace{\mathbb{E}_{p_{\text{data}}(\mathbf{x})} D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{constant w.r.t. } \theta}.$$

# From ELBO to Squared Error

Fix the encoder  $q_\phi(\mathbf{z}|\mathbf{x})$ , prior  $p(\mathbf{z})$  and decoder variance  $\sigma^2 > 0$ ; optimize only  $\theta$ .

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{p_{\text{data}}(\mathbf{x}) q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) - \underbrace{\mathbb{E}_{p_{\text{data}}(\mathbf{x})} D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{constant w.r.t. } \theta}.$$

The **conditional KL is constant in  $\theta$** , not necessarily zero. At  $p = q_{\text{agg}, \phi}$ :

$$\underbrace{D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z}))}_{\text{Marginal KL}} = 0, \quad \underbrace{\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z}))}_{\text{Average conditional KL}} = \mathbb{I}_q[\mathbf{x}, \mathbf{z}].$$

# From ELBO to Squared Error

Fix the encoder  $q_\phi(\mathbf{z}|\mathbf{x})$ , prior  $p(\mathbf{z})$  and decoder variance  $\sigma^2 > 0$ ; optimize only  $\theta$ .

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{p_{\text{data}}(\mathbf{x}) q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) - \underbrace{\mathbb{E}_{p_{\text{data}}(\mathbf{x})} D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{constant w.r.t. } \theta}.$$

The **conditional KL is constant in  $\theta$** , not necessarily zero. At  $p = q_{\text{agg}, \phi}$ :

$$\underbrace{D_{\text{KL}}(q_{\text{agg}, \phi}(\mathbf{z}) \| p(\mathbf{z}))}_{\text{Marginal KL}} = 0, \quad \underbrace{\frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}_i) \| p(\mathbf{z}))}_{\text{Average conditional KL}} = \mathbb{I}_q[\mathbf{x}, \mathbf{z}].$$

## Gaussian Decoder: Squared Reconstruction Error

$$p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{z}), \sigma^2 \mathbf{I}) \quad \Rightarrow \quad \log p_\theta(\mathbf{x}|\mathbf{z}) = -\frac{\|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|_2^2}{2\sigma^2} + \text{const.}$$

$$\arg \max_{\theta} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathcal{L}_{\phi, \theta}(\mathbf{x}) = \arg \min_{\theta} \mathbb{E}_{p_{\text{data}}(\mathbf{x}) q_\phi(\mathbf{z}|\mathbf{x})} \|\mathbf{x} - \boldsymbol{\mu}_\theta(\mathbf{z})\|_2^2.$$

# Why are VAE Generations Blurry?

Keep the encoder, prior and Gaussian decoder variance fixed. Maximizing ELBO gives

$$\arg \min_{\theta} \mathbb{E}_{p_{\text{data}}(\mathbf{x})q_{\phi}(\mathbf{z}|\mathbf{x})} \|\mathbf{x} - \boldsymbol{\mu}_{\theta}(\mathbf{z})\|_2^2.$$

# Why are VAE Generations Blurry?

Keep the encoder, prior and Gaussian decoder variance fixed. Maximizing ELBO gives

$$\arg \min_{\theta} \mathbb{E}_{p_{\text{data}}(\mathbf{x})q_{\phi}(\mathbf{z}|\mathbf{x})} \|\mathbf{x} - \boldsymbol{\mu}_{\theta}(\mathbf{z})\|_2^2.$$

## Optimal Decoder Mean

The optimal decoder mean is the **conditional expectation**:

$$\boldsymbol{\mu}^*(\mathbf{z}) = \mathbb{E}_{q_{\phi}(\mathbf{x}|\mathbf{z})}[\mathbf{x}] = \frac{\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[q_{\phi}(\mathbf{z}|\mathbf{x}) \cdot \mathbf{x}]}{\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[q_{\phi}(\mathbf{z}|\mathbf{x})]}$$

# Why are VAE Generations Blurry?

Keep the encoder, prior and Gaussian decoder variance fixed. Maximizing ELBO gives

$$\arg \min_{\theta} \mathbb{E}_{p_{\text{data}}(\mathbf{x})q_{\phi}(\mathbf{z}|\mathbf{x})} \|\mathbf{x} - \boldsymbol{\mu}_{\theta}(\mathbf{z})\|_2^2.$$

## Optimal Decoder Mean

The optimal decoder mean is the **conditional expectation**:

$$\boldsymbol{\mu}^*(\mathbf{z}) = \mathbb{E}_{q_{\phi}(\mathbf{x}|\mathbf{z})}[\mathbf{x}] = \frac{\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[q_{\phi}(\mathbf{z}|\mathbf{x}) \cdot \mathbf{x}]}{\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[q_{\phi}(\mathbf{z}|\mathbf{x})]}$$

## Blurriness from Averaging

- $p_{\text{data}}(\mathbf{x})$  and  $q_{\phi}(\mathbf{z}|\mathbf{x})$  give us the aggregated posterior  $q_{\text{agg},\phi}(\mathbf{z})$ .
- If  $\mathbf{x} \neq \mathbf{x}'$  map to overlapping latent regions,  $\boldsymbol{\mu}^*(\mathbf{z})$  averages over unrelated inputs.
- This leads to **blurry, non-distinct outputs**.

# Outline

01 ELBO Surgery and Optimal VAE Prior

## 02 Discrete Latent VAEs

**Discrete VAE Latent Representations**

Vector Quantized VAE (VQ-VAE)

03 Likelihood-Free Learning

04 Generative Adversarial Networks (GAN)

# Discrete VAE Latents

## Motivation

- VAE has used **continuous** latent variables  $\mathbf{z}$ .
- For some modalities, **discrete** representations  $\mathbf{z}$  may be a more natural choice.
- Advanced autoregressive models are highly effective for distributions over discrete variables (for example, transformers process discrete tokens).

# Discrete VAE Latents

## Motivation

- VAE has used **continuous** latent variables  $\mathbf{z}$ .
- For some modalities, **discrete** representations  $\mathbf{z}$  may be a more natural choice.
- Advanced autoregressive models are highly effective for distributions over discrete variables (for example, transformers process discrete tokens).

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \rightarrow \max_{\phi, \theta}.$$

# Discrete VAE Latents

## Motivation

- VAE has used **continuous** latent variables  $\mathbf{z}$ .
- For some modalities, **discrete** representations  $\mathbf{z}$  may be a more natural choice.
- Advanced autoregressive models are highly effective for distributions over discrete variables (for example, transformers process discrete tokens).

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \rightarrow \max_{\phi, \theta}.$$

- Apply the reparametrization trick to obtain unbiased gradients.
- Use Gaussian distributions for  $q_{\phi}(\mathbf{z}|\mathbf{x})$  and  $p(\mathbf{z})$  to compute the KL analytically.

# Discrete VAE Latents

## Assumptions

- Let  $c \sim \text{Cat}(\boldsymbol{\pi})$ , where

$$\boldsymbol{\pi} = (\pi_1, \dots, \pi_K), \quad \pi_k = P(c = k), \quad \sum_{k=1}^K \pi_k = 1.$$

- For tokenizer training, fix  $p(c) = \text{Uniform}\{1, \dots, K\}$ .

# Discrete VAE Latents

## Assumptions

- Let  $c \sim \text{Cat}(\boldsymbol{\pi})$ , where

$$\boldsymbol{\pi} = (\pi_1, \dots, \pi_K), \quad \pi_k = P(c = k), \quad \sum_{k=1}^K \pi_k = 1.$$

- For tokenizer training, fix  $p(c) = \text{Uniform}\{1, \dots, K\}$ .

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(c|\mathbf{x})} \log p_{\theta}(\mathbf{x}|c) - D_{\text{KL}}(q_{\phi}(c|\mathbf{x}) \| p(c)) \rightarrow \max_{\phi, \theta}.$$

$$D_{\text{KL}}(q_{\phi}(c|\mathbf{x}) \| p(c)) = \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log \frac{q_{\phi}(k|\mathbf{x})}{p(k)}$$

# Discrete VAE Latents

## Assumptions

- Let  $c \sim \text{Cat}(\boldsymbol{\pi})$ , where

$$\boldsymbol{\pi} = (\pi_1, \dots, \pi_K), \quad \pi_k = P(c = k), \quad \sum_{k=1}^K \pi_k = 1.$$

- For tokenizer training, fix  $p(c) = \text{Uniform}\{1, \dots, K\}$ .

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(c|\mathbf{x})} \log p_{\theta}(\mathbf{x}|c) - D_{\text{KL}}(q_{\phi}(c|\mathbf{x}) \| p(c)) \rightarrow \max_{\phi, \theta}.$$

$$\begin{aligned} D_{\text{KL}}(q_{\phi}(c|\mathbf{x}) \| p(c)) &= \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log \frac{q_{\phi}(k|\mathbf{x})}{p(k)} \\ &= \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log q_{\phi}(k|\mathbf{x}) - \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log p(k) \end{aligned}$$

# Discrete VAE Latents

## Assumptions

- Let  $c \sim \text{Cat}(\boldsymbol{\pi})$ , where

$$\boldsymbol{\pi} = (\pi_1, \dots, \pi_K), \quad \pi_k = P(c = k), \quad \sum_{k=1}^K \pi_k = 1.$$

- For tokenizer training, fix  $p(c) = \text{Uniform}\{1, \dots, K\}$ .

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(c|\mathbf{x})} \log p_{\theta}(\mathbf{x}|c) - D_{\text{KL}}(q_{\phi}(c|\mathbf{x}) \| p(c)) \rightarrow \max_{\phi, \theta}.$$

$$\begin{aligned} D_{\text{KL}}(q_{\phi}(c|\mathbf{x}) \| p(c)) &= \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log \frac{q_{\phi}(k|\mathbf{x})}{p(k)} \\ &= \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log q_{\phi}(k|\mathbf{x}) - \sum_{k=1}^K q_{\phi}(k|\mathbf{x}) \log p(k) \\ &= -H(q_{\phi}(c|\mathbf{x})) + \log K. \end{aligned}$$

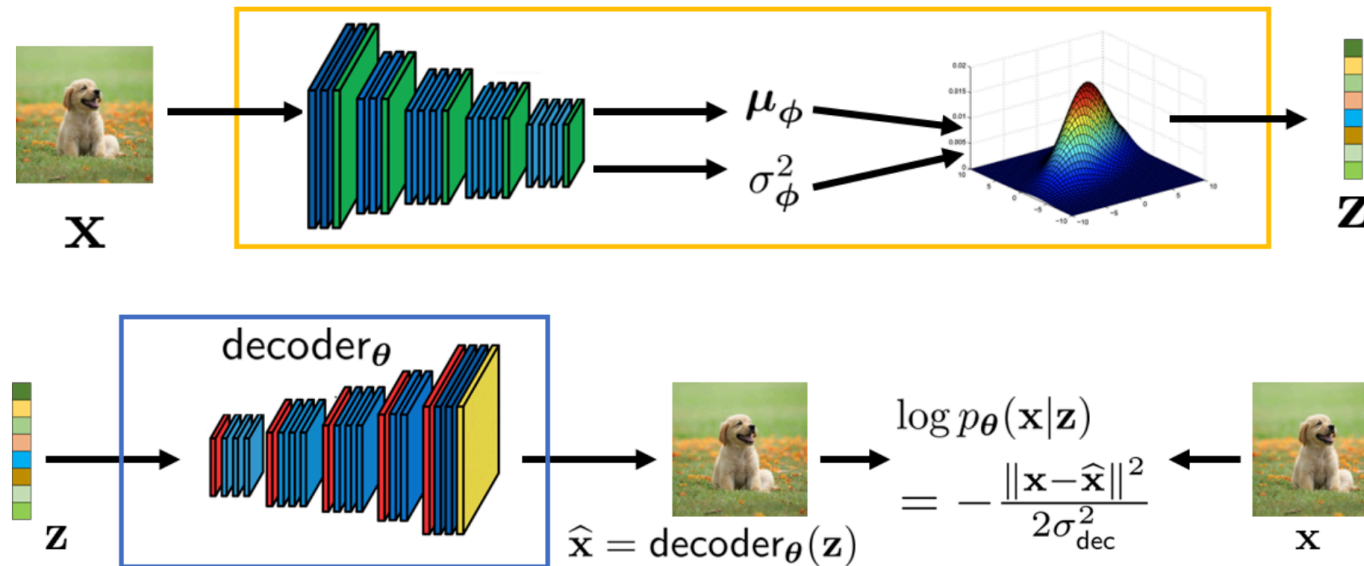
# Discrete VAE Latents

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(c|\mathbf{x})} \log p_{\theta}(\mathbf{x}|c) + H(q_{\phi}(c|\mathbf{x})) - \log K \rightarrow \max_{\phi, \theta}.$$

# Discrete VAE Latents

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(c|\mathbf{x})} \log p_{\theta}(\mathbf{x}|c) + H(q_{\phi}(c|\mathbf{x})) - \log K \rightarrow \max_{\phi, \theta}.$$

- The encoder should output a discrete distribution  $q_{\phi}(c|\mathbf{x})$ .
- We need an analogue of the reparametrization trick for discrete  $q_{\phi}(c|\mathbf{x})$ .
- The decoder  $p_{\theta}(\mathbf{x}|c)$  must take a discrete random variable  $c$  as input.



# Outline

01 ELBO Surgery and Optimal VAE Prior

## 02 Discrete Latent VAEs

Discrete VAE Latent Representations

**Vector Quantized VAE (VQ-VAE)**

03 Likelihood-Free Learning

04 Generative Adversarial Networks (GAN)

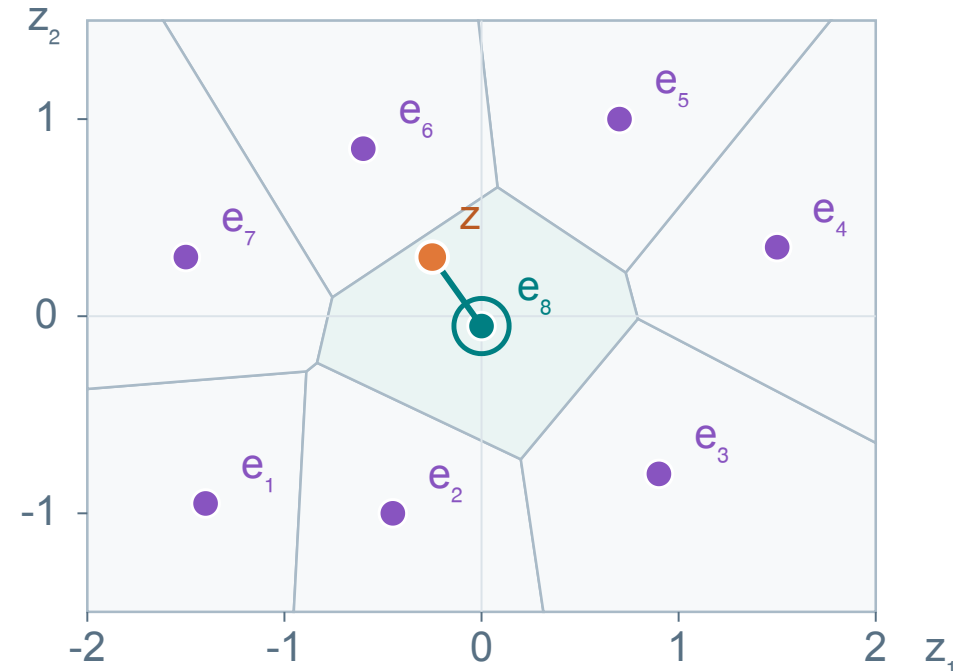
# Vector Quantization

A **codebook** contains  $K$  vectors:  $\{\mathbf{e}_k\}_{k=1}^K$ , with  $\mathbf{e}_k \in \mathbb{R}^L$ .

# Vector Quantization

A **codebook** contains  $K$  vectors:  $\{\mathbf{e}_k\}_{k=1}^K$ , with  $\mathbf{e}_k \in \mathbb{R}^L$ .

$$\mathbf{z}_q = \mathbf{q}(\mathbf{z}) = \mathbf{e}_{k^*}, \quad \text{where } k^* = \arg \min_k \|\mathbf{z} - \mathbf{e}_k\|.$$



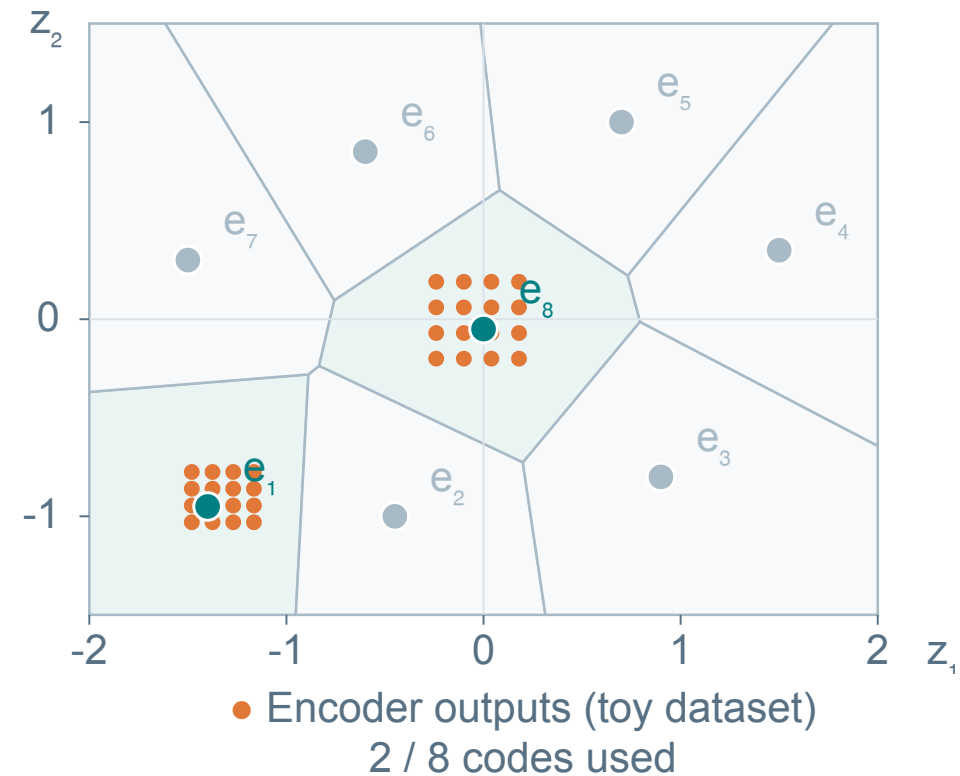
# Vector Quantization

A **codebook** contains  $K$  vectors:  $\{\mathbf{e}_k\}_{k=1}^K$ , with  $\mathbf{e}_k \in \mathbb{R}^L$ .

$$\mathbf{z}_q = \mathbf{q}(\mathbf{z}) = \mathbf{e}_{k^*}, \quad \text{where } k^* = \arg \min_k \|\mathbf{z} - \mathbf{e}_k\|.$$

## Codebook Collapse

Only a small subset of entries is used across the data.



# Vector Quantization

A **codebook** contains  $K$  vectors:  $\{\mathbf{e}_k\}_{k=1}^K$ , with  $\mathbf{e}_k \in \mathbb{R}^L$ .

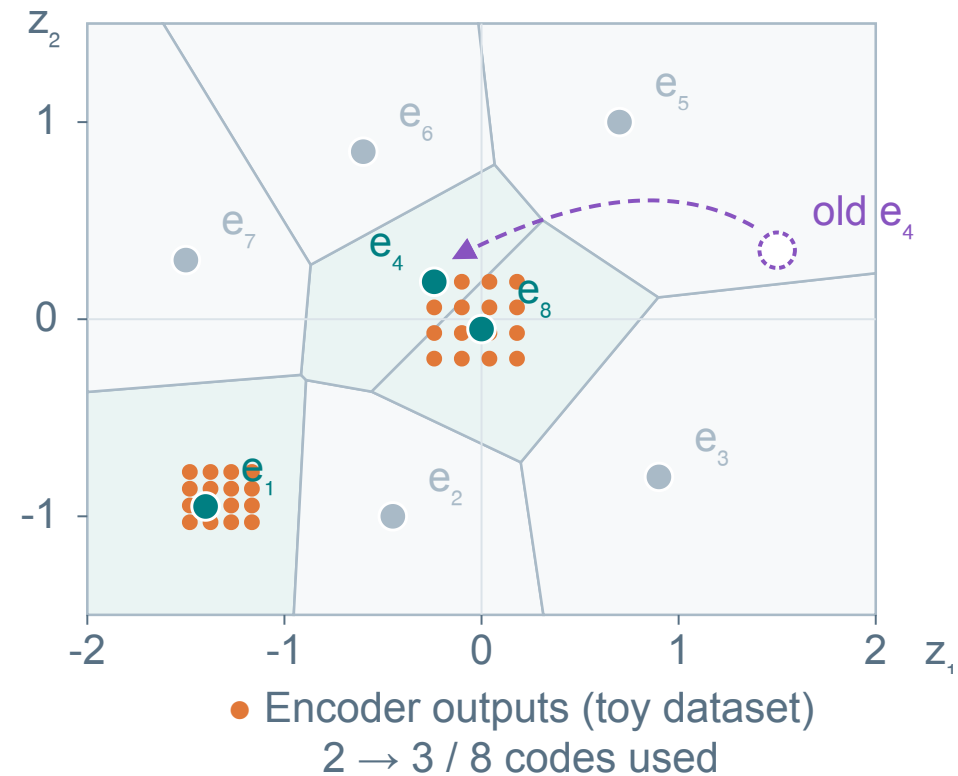
$$\mathbf{z}_q = \mathbf{q}(\mathbf{z}) = \mathbf{e}_{k^*}, \quad \text{where } k^* = \arg \min_k \|\mathbf{z} - \mathbf{e}_k\|.$$

## Codebook Collapse

Only a small subset of entries is used across the data.

## Restart an Unused Code

Move it to an encoder output; assignments and Voronoi regions change.



# Vector Quantized VAE (VQ-VAE)

- The encoder outputs a continuous vector  $\mathbf{z}_e = \text{NN}_{e,\phi}(\mathbf{x}) \in \mathbb{R}^L$ .
- Quantization **deterministically** maps  $\mathbf{z}_e$  to its quantized codebook vector  $\mathbf{z}_q$ .
- The decoder is conditioned on codebook entries  $\mathbf{e}_c$ , i.e., via  $p_\theta(\mathbf{x}|\mathbf{e}_c)$  (instead of  $p_\theta(\mathbf{x}|c)$ ).

# Vector Quantized VAE (VQ-VAE)

- The encoder outputs a continuous vector  $\mathbf{z}_e = \text{NN}_{e,\phi}(\mathbf{x}) \in \mathbb{R}^L$ .
- Quantization **deterministically** maps  $\mathbf{z}_e$  to its quantized codebook vector  $\mathbf{z}_q$ .
- The decoder is conditioned on codebook entries  $\mathbf{e}_c$ , i.e., via  $p_\theta(\mathbf{x}|\mathbf{e}_c)$  (instead of  $p_\theta(\mathbf{x}|c)$ ).

## Deterministic Variational Posterior

$$q_\phi(c = k^*|\mathbf{x}) = \begin{cases} 1, & \text{for } k^* = \arg \min_k \|\mathbf{z}_e - \mathbf{e}_k\|; \\ 0, & \text{otherwise.} \end{cases}$$

# Vector Quantized VAE (VQ-VAE)

- The encoder outputs a continuous vector  $\mathbf{z}_e = \text{NN}_{e,\phi}(\mathbf{x}) \in \mathbb{R}^L$ .
- Quantization **deterministically** maps  $\mathbf{z}_e$  to its quantized codebook vector  $\mathbf{z}_q$ .
- The decoder is conditioned on codebook entries  $\mathbf{e}_c$ , i.e., via  $p_\theta(\mathbf{x}|\mathbf{e}_c)$  (instead of  $p_\theta(\mathbf{x}|c)$ ).

## Deterministic Variational Posterior

$$q_\phi(c = k^*|\mathbf{x}) = \begin{cases} 1, & \text{for } k^* = \arg \min_k \|\mathbf{z}_e - \mathbf{e}_k\|; \\ 0, & \text{otherwise.} \end{cases}$$

$$D_{\text{KL}}(q_\phi(c|\mathbf{x})\|p(c)) = -\underbrace{\text{H}(q_\phi(c|\mathbf{x}))}_{=0} + \log K = \log K.$$

# Vector Quantized VAE (VQ-VAE)

- The encoder outputs a continuous vector  $\mathbf{z}_e = \text{NN}_{e,\phi}(\mathbf{x}) \in \mathbb{R}^L$ .
- Quantization **deterministically** maps  $\mathbf{z}_e$  to its quantized codebook vector  $\mathbf{z}_q$ .
- The decoder is conditioned on codebook entries  $\mathbf{e}_c$ , i.e., via  $p_\theta(\mathbf{x}|\mathbf{e}_c)$  (instead of  $p_\theta(\mathbf{x}|c)$ ).

## Deterministic Variational Posterior

$$q_\phi(c = k^*|\mathbf{x}) = \begin{cases} 1, & \text{for } k^* = \arg \min_k \|\mathbf{z}_e - \mathbf{e}_k\|; \\ 0, & \text{otherwise.} \end{cases}$$

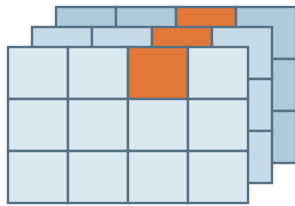
$$D_{\text{KL}}(q_\phi(c|\mathbf{x})\|p(c)) = - \underbrace{\text{H}(q_\phi(c|\mathbf{x}))}_{=0} + \log K = \log K.$$

**Constant KL:** The aggregated posterior is not pushed toward the uniform prior. We therefore fit a prior to it in a second stage.

# Vector Quantized VAE (VQ-VAE): Forward

Quantize each of the  $M = WH$  vectors using the **same codebook**.

Encoder outputs



$$\mathbf{z}_e \in \mathbb{R}^{W \times H \times L}$$

Nearest code



Code map

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 4 | 4 |
| 1 | 2 | 4 | 3 |
| 2 | 2 | 3 | 3 |

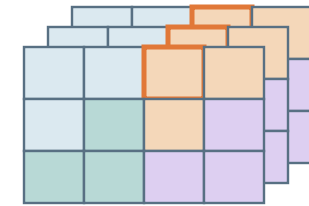
$$\mathbf{c} \in \{1, \dots, K\}^{W \times H}$$

Lookup



$$\mathbf{e}_{c_{ij}}$$

Decoder inputs

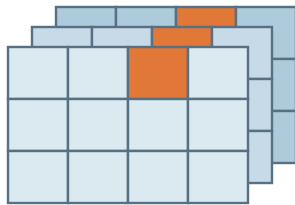


$$\mathbf{z}_q \in \mathbb{R}^{W \times H \times L}$$

# Vector Quantized VAE (VQ-VAE): Forward

Quantize each of the  $M = WH$  vectors using the **same codebook**.

Encoder outputs



$$\mathbf{z}_e \in \mathbb{R}^{W \times H \times L}$$

Nearest code



Code map

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 4 | 4 |
| 1 | 2 | 4 | 3 |
| 2 | 2 | 3 | 3 |

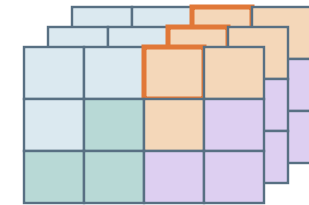
$$\mathbf{c} \in \{1, \dots, K\}^{W \times H}$$

Lookup



$$\mathbf{e}_{c_{ij}}$$

Decoder inputs



$$\mathbf{z}_q \in \mathbb{R}^{W \times H \times L}$$

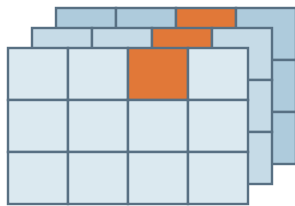
## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{c}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}_q(\mathbf{c})) - M \log K$$

# Vector Quantized VAE (VQ-VAE): Forward

Quantize each of the  $M = WH$  vectors using the **same codebook**.

Encoder outputs



$$\mathbf{z}_e \in \mathbb{R}^{W \times H \times L}$$

Nearest code



Code map

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 4 | 4 |
| 1 | 2 | 4 | 3 |
| 2 | 2 | 3 | 3 |

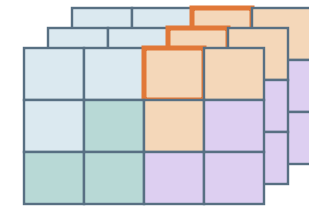
$$\mathbf{c} \in \{1, \dots, K\}^{W \times H}$$

Lookup



$$\mathbf{e}_{c_{ij}}$$

Decoder inputs



$$\mathbf{z}_q \in \mathbb{R}^{W \times H \times L}$$

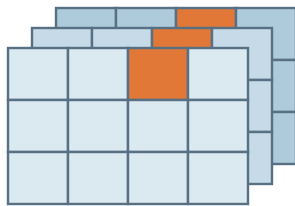
## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{c}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}_q(\mathbf{c})) - M \log K = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K.$$

# Vector Quantized VAE (VQ-VAE): Forward

Quantize each of the  $M = WH$  vectors using the **same codebook**.

Encoder outputs



$$\mathbf{z}_e \in \mathbb{R}^{W \times H \times L}$$

Nearest code



Code map

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 4 | 4 |
| 1 | 2 | 4 | 3 |
| 2 | 2 | 3 | 3 |

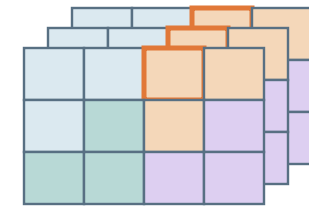
$$\mathbf{c} \in \{1, \dots, K\}^{W \times H}$$

Lookup



$$\mathbf{e}_{c_{ij}}$$

Decoder inputs



$$\mathbf{z}_q \in \mathbb{R}^{W \times H \times L}$$

## ELBO

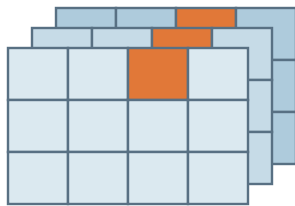
$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{c}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}_q(\mathbf{c})) - M \log K = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K.$$

The deterministic map and independent uniform training prior give  $D_{\text{KL}} = M \log K$ .

# Vector Quantized VAE (VQ-VAE): Forward

Quantize each of the  $M = WH$  vectors using the **same codebook**.

Encoder outputs



$$\mathbf{z}_e \in \mathbb{R}^{W \times H \times L}$$

Nearest code



Code map

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 4 | 4 |
| 1 | 2 | 4 | 3 |
| 2 | 2 | 3 | 3 |

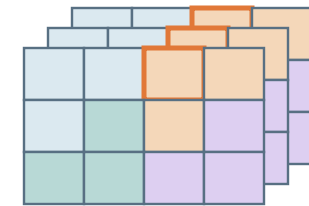
$$\mathbf{c} \in \{1, \dots, K\}^{W \times H}$$

Lookup



$$\mathbf{e}_{c_{ij}}$$

Decoder inputs



$$\mathbf{z}_q \in \mathbb{R}^{W \times H \times L}$$

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{c}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}_q(\mathbf{c})) - M \log K = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K.$$

The deterministic map and independent uniform training prior give  $D_{\text{KL}} = M \log K$ .

**Challenge:** The  $\arg \min$  operation is non-differentiable.

# Vector Quantized VAE (VQ-VAE): Backward

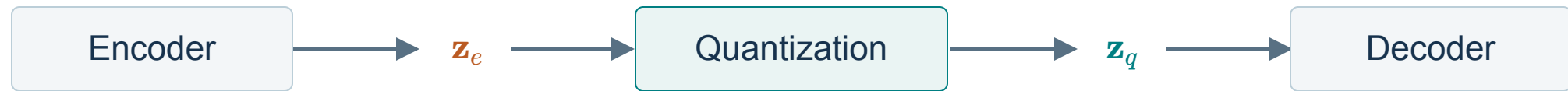
## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K, \quad \mathbf{z}_{q,m} = \mathbf{e}_{c_m}, \quad m = 1, \dots, M.$$

# Vector Quantized VAE (VQ-VAE): Backward

## ELBO

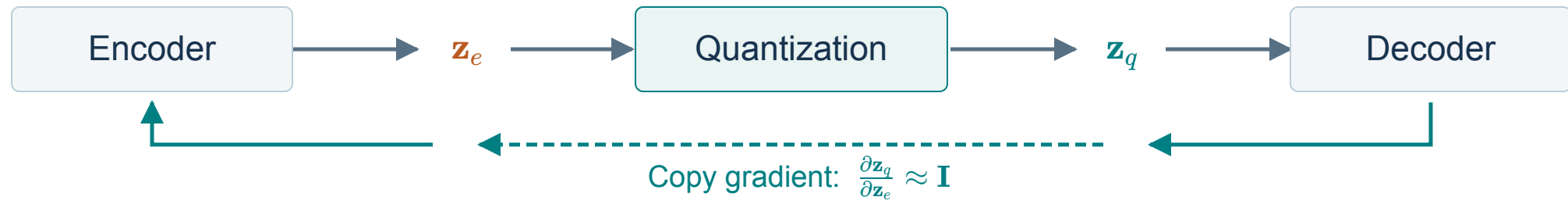
$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K, \quad \mathbf{z}_{q,m} = \mathbf{e}_{c_m}, \quad m = 1, \dots, M.$$



# Vector Quantized VAE (VQ-VAE): Backward

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K, \quad \mathbf{z}_{q,m} = \mathbf{e}_{c_m}, \quad m = 1, \dots, M.$$



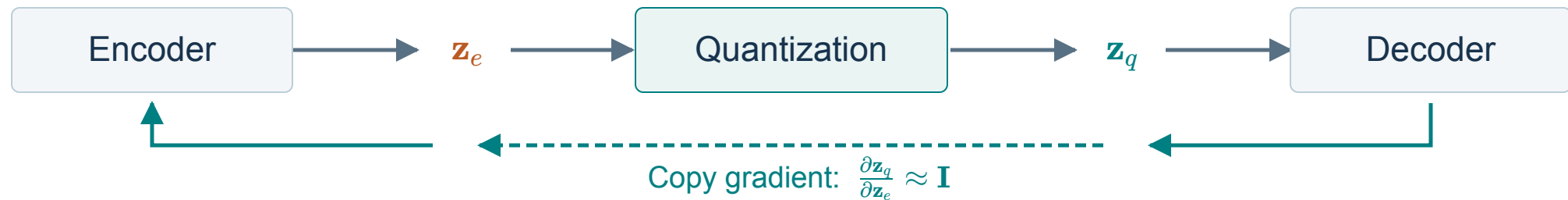
## Straight-Through Gradient Estimator

$$\frac{\partial \log p(\mathbf{x}|\mathbf{z}_q, \theta)}{\partial \phi} = \frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_q}{\partial \phi}$$

# Vector Quantized VAE (VQ-VAE): Backward

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K, \quad \mathbf{z}_{q,m} = \mathbf{e}_{c_m}, \quad m = 1, \dots, M.$$



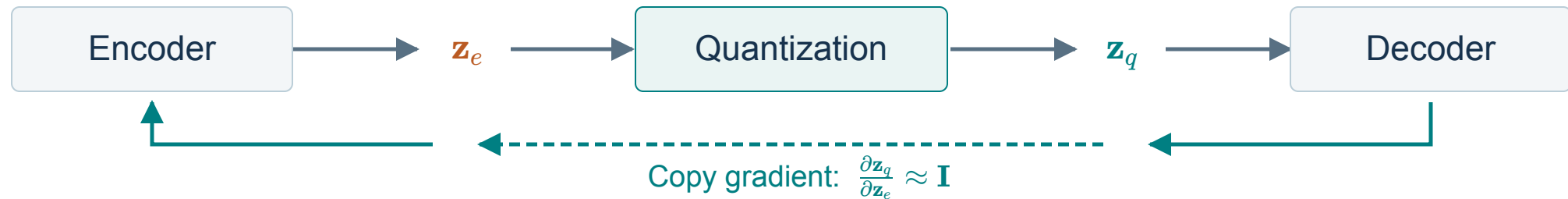
## Straight-Through Gradient Estimator

$$\begin{aligned} \frac{\partial \log p(\mathbf{x}|\mathbf{z}_q, \theta)}{\partial \phi} &= \frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_q}{\partial \phi} \\ &= \frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_q}{\partial \mathbf{z}_e} \cdot \frac{\partial \mathbf{z}_e}{\partial \phi} \end{aligned}$$

# Vector Quantized VAE (VQ-VAE): Backward

## ELBO

$$\mathcal{L}_{\phi, \theta}(\mathbf{x}) = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K, \quad \mathbf{z}_{q,m} = \mathbf{e}_{c_m}, \quad m = 1, \dots, M.$$



## Straight-Through Gradient Estimator

$$\begin{aligned} \frac{\partial \log p(\mathbf{x}|\mathbf{z}_q, \theta)}{\partial \phi} &= \frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_q}{\partial \phi} \\ &= \frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_q}{\partial \mathbf{z}_e} \cdot \frac{\partial \mathbf{z}_e}{\partial \phi} \\ &\approx \frac{\partial \log p_{\theta}(\mathbf{x}|\mathbf{z}_q)}{\partial \mathbf{z}_q} \cdot \frac{\partial \mathbf{z}_e}{\partial \phi} \end{aligned}$$

# VQ-VAE: Learning a Prior over Codes

The **tokenizer** (encoder + quantizer) produces  $M = WH$  codes:  $\mathbf{c} = (c_1, \dots, c_M)$ .

**Target: Aggregated Posterior (ELBO Surgery)**

$$p_{\psi}(\mathbf{c}) \approx q_{\text{agg}, \phi}(\mathbf{c}) = \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{c} | \mathbf{x}_i).$$

# VQ-VAE: Learning a Prior over Codes

The **tokenizer** (encoder + quantizer) produces  $M = WH$  codes:  $\mathbf{c} = (c_1, \dots, c_M)$ .

**Target: Aggregated Posterior (ELBO Surgery)**

$$p_{\psi}(\mathbf{c}) \approx q_{\text{agg}, \phi}(\mathbf{c}) = \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{c} | \mathbf{x}_i).$$

## Training

1. Fix the trained tokenizer and decoder.
2. Encode the training images  $\mathbf{x}_i$  into code maps  $\mathbf{c}_i$ .
3. Fit the **autoregressive prior** (Lecture 1) by maximum likelihood:

$$\sum_{i=1}^n \log p_{\psi}(\mathbf{c}_i) = \sum_{i=1}^n \sum_{m=1}^M \log p_{\psi}(c_{i,m} | c_{i,1}, \dots, c_{i,m-1}) \rightarrow \max_{\psi}.$$

# VQ-VAE: Learning a Prior over Codes

The **tokenizer** (encoder + quantizer) produces  $M = WH$  codes:  $\mathbf{c} = (c_1, \dots, c_M)$ .

**Target: Aggregated Posterior (ELBO Surgery)**

$$p_{\psi}(\mathbf{c}) \approx q_{\text{agg}, \phi}(\mathbf{c}) = \frac{1}{n} \sum_{i=1}^n q_{\phi}(\mathbf{c} | \mathbf{x}_i).$$

## Training

1. Fix the trained tokenizer and decoder.
2. Encode the training images  $\mathbf{x}_i$  into code maps  $\mathbf{c}_i$ .
3. Fit the **autoregressive prior** (Lecture 1) by maximum likelihood:

$$\sum_{i=1}^n \log p_{\psi}(\mathbf{c}_i) = \sum_{i=1}^n \sum_{m=1}^M \log p_{\psi}(c_{i,m} | c_{i,1}, \dots, c_{i,m-1}) \rightarrow \max_{\psi}.$$

$p_{\psi}(\mathbf{c}) \rightarrow$  **code map**  $\rightarrow$  **codebook lookup**  $\rightarrow$  **decoder**  $\rightarrow \mathbf{x}$ .

# Vector Quantized VAE-2 (VQ-VAE-2)

VQ-VAE-2 combines code maps at **multiple resolutions**.

**Sample Diversity: Learned Autoregressive Priors**



**VQ-VAE (Proposed)**

**BigGAN deep**

# Vector Quantized VAE (VQ-VAE): Final algorithm

## Training

### Stage 1: tokenizer.

1. Sample  $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ .
2. Compute encoder outputs  $\mathbf{z}_{e,m} = \text{NN}_{e,\phi}(\mathbf{x})_m$ .
3. Compute the quantized representation at each location:

$$c_m = \arg \min_k \|\mathbf{z}_{e,m} - \mathbf{e}_k\|_2, \quad \mathbf{z}_{q,m} = \mathbf{e}_{c_m}.$$

4. Compute the ELBO with the uniform training prior:

$$\mathcal{L}_{\phi,\theta}(\mathbf{x}) = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K.$$

5. Compute the loss with codebook and commitment terms:

$$\begin{aligned} \mathcal{L} = & -\mathcal{L}_{\phi,\theta}(\mathbf{x}) + \sum_{m=1}^M \|\text{sg}[\mathbf{z}_{e,m}] - \mathbf{e}_{c_m}\|_2^2 \\ & + \beta \sum_{m=1}^M \|\mathbf{z}_{e,m} - \text{sg}[\mathbf{e}_{c_m}]\|_2^2. \end{aligned}$$

6. Update  $\phi$ ,  $\theta$  and  $\{\mathbf{e}_k\}$  (straight-through for  $\phi$ ).

# Vector Quantized VAE (VQ-VAE): Final algorithm

## Training

### Stage 1: tokenizer.

1. Sample  $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ .
2. Compute encoder outputs  $\mathbf{z}_{e,m} = \text{NN}_{e,\phi}(\mathbf{x})_m$ .
3. Compute the quantized representation at each location:

$$c_m = \arg \min_k \|\mathbf{z}_{e,m} - \mathbf{e}_k\|_2, \quad \mathbf{z}_{q,m} = \mathbf{e}_{c_m}.$$

4. Compute the ELBO with the uniform training prior:

$$\mathcal{L}_{\phi,\theta}(\mathbf{x}) = \log p_{\theta}(\mathbf{x}|\mathbf{z}_q) - M \log K.$$

5. Compute the loss with codebook and commitment terms:

$$\begin{aligned} \mathcal{L} = & -\mathcal{L}_{\phi,\theta}(\mathbf{x}) + \sum_{m=1}^M \|\text{sg}[\mathbf{z}_{e,m}] - \mathbf{e}_{c_m}\|_2^2 \\ & + \beta \sum_{m=1}^M \|\mathbf{z}_{e,m} - \text{sg}[\mathbf{e}_{c_m}]\|_2^2. \end{aligned}$$

6. Update  $\phi$ ,  $\theta$  and  $\{\mathbf{e}_k\}$  (straight-through for  $\phi$ ).

## Sampling

After **Stage 2: prior training**, fix all parameters.

1. Sample the code map  $\mathbf{c} \sim p_{\psi}(\mathbf{c})$  autoregressively.
2. Compute  $\mathbf{z}_{q,m} = \mathbf{e}_{c_m}$  at each location.
3. Sample  $\mathbf{x} \sim p_{\theta}(\mathbf{x}|\mathbf{z}_q)$ .

# Outline

01 ELBO Surgery and Optimal VAE Prior

02 Discrete Latent VAEs

Discrete VAE Latent Representations  
Vector Quantized VAE (VQ-VAE)

03 **Likelihood-Free Learning**

04 Generative Adversarial Networks (GAN)

# Likelihood-Based Models

**Poor Likelihood**  
**High-Quality Samples**

$$p_1(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \mathcal{N}(\mathbf{x} | \mathbf{x}_i, \epsilon \mathbf{I})$$

If  $\epsilon$  is very small, this model produces excellent, sharp samples but achieves poor likelihoods on test data.

# Likelihood-Based Models

## Poor Likelihood High-Quality Samples

$$p_1(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \mathcal{N}(\mathbf{x} | \mathbf{x}_i, \epsilon \mathbf{I})$$

If  $\epsilon$  is very small, this model produces excellent, sharp samples but achieves poor likelihoods on test data.

## High Likelihood Poor Samples

$$\begin{aligned} p_2(\mathbf{x}) &= 0.01p(\mathbf{x}) + 0.99p_{\text{noise}}(\mathbf{x}) \\ \log [0.01p(\mathbf{x}) + 0.99p_{\text{noise}}(\mathbf{x})] \\ &\geq \log[0.01p(\mathbf{x})] \\ &= \log p(\mathbf{x}) - \log 100 \end{aligned}$$

This model contains mostly noisy, irrelevant samples; for high dimensions,  $\log p(\mathbf{x})$  scales linearly with  $m$ .

# Likelihood-Based Models

## Poor Likelihood High-Quality Samples

$$p_1(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \mathcal{N}(\mathbf{x} | \mathbf{x}_i, \epsilon \mathbf{I})$$

If  $\epsilon$  is very small, this model produces excellent, sharp samples but achieves poor likelihoods on test data.

## High Likelihood Poor Samples

$$\begin{aligned} p_2(\mathbf{x}) &= 0.01p(\mathbf{x}) + 0.99p_{\text{noise}}(\mathbf{x}) \\ \log [0.01p(\mathbf{x}) + 0.99p_{\text{noise}}(\mathbf{x})] \\ &\geq \log[0.01p(\mathbf{x})] \\ &= \log p(\mathbf{x}) - \log 100 \end{aligned}$$

This model contains mostly noisy, irrelevant samples; for high dimensions,  $\log p(\mathbf{x})$  scales linearly with  $m$ .

- Likelihood isn't always a suitable metric for evaluating generative models.
- Sometimes, the likelihood function can't even be computed exactly.

# Likelihood-Free Learning

## Motivation

We're interested in approximating the true data distribution  $p_{\text{data}}(\mathbf{x})$ . Instead of searching over all distributions, let's learn a model  $p_{\theta}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$ .

# Likelihood-Free Learning

## Motivation

We're interested in approximating the true data distribution  $p_{\text{data}}(\mathbf{x})$ . Instead of searching over all distributions, let's learn a model  $p_{\theta}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$ .

Suppose we have two sets of samples:

- $\{\mathbf{x}_i\}_{i=1}^{n_1} \sim p_{\text{data}}(\mathbf{x})$  — real data;
- $\{\mathbf{x}_i\}_{i=1}^{n_2} \sim p_{\theta}(\mathbf{x})$  — generated (fake) data.

# Likelihood-Free Learning

## Motivation

We're interested in approximating the true data distribution  $p_{\text{data}}(\mathbf{x})$ . Instead of searching over all distributions, let's learn a model  $p_{\theta}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$ .

Suppose we have two sets of samples:

- $\{\mathbf{x}_i\}_{i=1}^{n_1} \sim p_{\text{data}}(\mathbf{x})$  — real data;
- $\{\mathbf{x}_i\}_{i=1}^{n_2} \sim p_{\theta}(\mathbf{x})$  — generated (fake) data.

Define a discriminative model (classifier):

$$p(y = 1|\mathbf{x}) = P(\mathbf{x} \sim p_{\text{data}}(\mathbf{x})); \quad p(y = 0|\mathbf{x}) = P(\mathbf{x} \sim p_{\theta}(\mathbf{x}))$$

# Likelihood-Free Learning

## Motivation

We're interested in approximating the true data distribution  $p_{\text{data}}(\mathbf{x})$ . Instead of searching over all distributions, let's learn a model  $p_{\theta}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$ .

Suppose we have two sets of samples:

- $\{\mathbf{x}_i\}_{i=1}^{n_1} \sim p_{\text{data}}(\mathbf{x})$  — real data;
- $\{\mathbf{x}_i\}_{i=1}^{n_2} \sim p_{\theta}(\mathbf{x})$  — generated (fake) data.

Define a discriminative model (classifier):

$$p(y = 1|\mathbf{x}) = P(\mathbf{x} \sim p_{\text{data}}(\mathbf{x})); \quad p(y = 0|\mathbf{x}) = P(\mathbf{x} \sim p_{\theta}(\mathbf{x}))$$

## Assumption

The generative model  $p_{\theta}(\mathbf{x})$  matches  $p_{\text{data}}(\mathbf{x})$  if a discriminative model  $p(y|\mathbf{x})$  can't distinguish between them — that is, if  $p(y = 1|\mathbf{x}) = 0.5$  for every  $\mathbf{x}$ .

# Outline

01 ELBO Surgery and Optimal VAE Prior

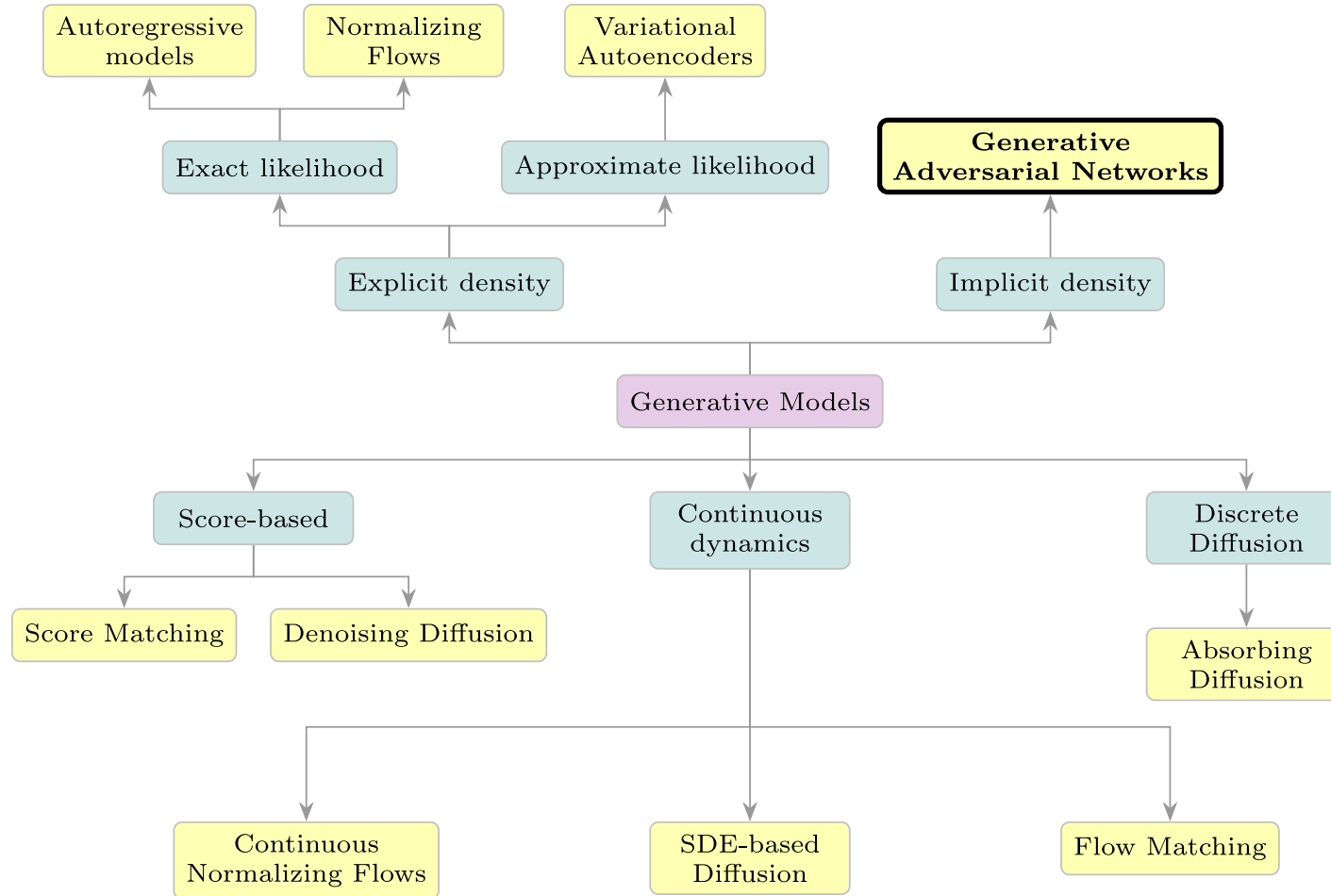
02 Discrete Latent VAEs

Discrete VAE Latent Representations  
Vector Quantized VAE (VQ-VAE)

03 Likelihood-Free Learning

**04 Generative Adversarial Networks (GAN)**

# Generative Models Taxonomy



# Generative Adversarial Networks (GAN)

- The more expressive the discriminator  $p(y|\mathbf{x})$ , the closer we get to the optimal  $p_\theta(\mathbf{x})$ .
- Standard classifiers are trained by minimizing cross-entropy loss  $-\mathbb{E}_{\hat{p}(\mathbf{x}, y)} \log p(y|\mathbf{x})$  with  $\hat{p}(\mathbf{x}, y) = \frac{1}{2}\mathbb{I}_{y=1}p_{\text{data}}(\mathbf{x}) + \frac{1}{2}\mathbb{I}_{y=0}p_\theta(\mathbf{x})$ .

# Generative Adversarial Networks (GAN)

- The more expressive the discriminator  $p(y|\mathbf{x})$ , the closer we get to the optimal  $p_\theta(\mathbf{x})$ .
- Standard classifiers are trained by minimizing cross-entropy loss  $-\mathbb{E}_{\hat{p}(\mathbf{x},y)} \log p(y|\mathbf{x})$  with  $\hat{p}(\mathbf{x}, y) = \frac{1}{2}\mathbb{I}_{y=1}p_{\text{data}}(\mathbf{x}) + \frac{1}{2}\mathbb{I}_{y=0}p_\theta(\mathbf{x})$ .

## Cross-Entropy for Discriminator

$$\min_{p(y|\mathbf{x})} \left[ -\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y=1|\mathbf{x}) - \mathbb{E}_{p_\theta(\mathbf{x})} \log p(y=0|\mathbf{x}) \right]$$

$$\max_{p(y|\mathbf{x})} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y=1|\mathbf{x}) + \mathbb{E}_{p_\theta(\mathbf{x})} \log p(y=0|\mathbf{x}) \right]$$

# Generative Adversarial Networks (GAN)

- The more expressive the discriminator  $p(y|\mathbf{x})$ , the closer we get to the optimal  $p_\theta(\mathbf{x})$ .
- Standard classifiers are trained by minimizing cross-entropy loss  $-\mathbb{E}_{\hat{p}(\mathbf{x}, y)} \log p(y|\mathbf{x})$  with  $\hat{p}(\mathbf{x}, y) = \frac{1}{2} \mathbb{I}_{y=1} p_{\text{data}}(\mathbf{x}) + \frac{1}{2} \mathbb{I}_{y=0} p_\theta(\mathbf{x})$ .

## Cross-Entropy for Discriminator

$$\min_{p(y|\mathbf{x})} \left[ -\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y=1|\mathbf{x}) - \mathbb{E}_{p_\theta(\mathbf{x})} \log p(y=0|\mathbf{x}) \right]$$

$$\max_{p(y|\mathbf{x})} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y=1|\mathbf{x}) + \mathbb{E}_{p_\theta(\mathbf{x})} \log p(y=0|\mathbf{x}) \right]$$

## Generative Model

Suppose  $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})$ , where  $p(\mathbf{z})$  is a base distribution, and  $p_\theta(\mathbf{x}|\mathbf{z}) = \delta(\mathbf{x} - \mathbf{G}_\theta(\mathbf{z}))$  is deterministic.

# Generative Adversarial Networks (GAN)

- The more expressive the discriminator  $p(y|\mathbf{x})$ , the closer we get to the optimal  $p_\theta(\mathbf{x})$ .
- Standard classifiers are trained by minimizing cross-entropy loss  $-\mathbb{E}_{\hat{p}(\mathbf{x},y)} \log p(y|\mathbf{x})$  with  $\hat{p}(\mathbf{x}, y) = \frac{1}{2}\mathbb{I}_{y=1}p_{\text{data}}(\mathbf{x}) + \frac{1}{2}\mathbb{I}_{y=0}p_\theta(\mathbf{x})$ .

## Cross-Entropy for Discriminator

$$\min_{p(y|\mathbf{x})} \left[ -\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y=1|\mathbf{x}) - \mathbb{E}_{p_\theta(\mathbf{x})} \log p(y=0|\mathbf{x}) \right]$$

$$\max_{p(y|\mathbf{x})} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y=1|\mathbf{x}) + \mathbb{E}_{p_\theta(\mathbf{x})} \log p(y=0|\mathbf{x}) \right]$$

## Generative Model

Suppose  $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})$ , where  $p(\mathbf{z})$  is a base distribution, and  $p_\theta(\mathbf{x}|\mathbf{z}) = \delta(\mathbf{x} - \mathbf{G}_\theta(\mathbf{z}))$  is deterministic.

**Connection to VAEs:** Both models sample  $\mathbf{z} \sim p(\mathbf{z})$ . A VAE then samples  $\mathbf{x} \sim p_\theta(\mathbf{x}|\mathbf{z})$ ; here, the decoder is **deterministic**:  $\mathbf{x} = \mathbf{G}_\theta(\mathbf{z})$ .

# Generative Adversarial Networks (GAN)

## Cross-Entropy for Discriminative Model

$$\max_{p(y|\mathbf{x})} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y = 1|\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log p(y = 0|\mathbf{x}) \right]$$

# Generative Adversarial Networks (GAN)

## Cross-Entropy for Discriminative Model

$$\max_{p(y|\mathbf{x})} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y = 1|\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log p(y = 0|\mathbf{x}) \right]$$

- **Discriminator:** A classifier  $p_{\phi}(y = 1|\mathbf{x}) = D_{\phi}(\mathbf{x}) \in [0, 1]$ , distinguishing real and generated samples. The discriminator aims to **maximize** cross-entropy.
- **Generator:** The generative model  $\mathbf{x} = \mathbf{G}_{\theta}(\mathbf{z})$ ,  $\mathbf{z} \sim p(\mathbf{z})$ , seeks to fool the discriminator. The generator aims to **minimize** cross-entropy.

# Generative Adversarial Networks (GAN)

## Cross-Entropy for Discriminative Model

$$\max_{p(y|\mathbf{x})} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y = 1|\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log p(y = 0|\mathbf{x}) \right]$$

- **Discriminator:** A classifier  $p_{\phi}(y = 1|\mathbf{x}) = D_{\phi}(\mathbf{x}) \in [0, 1]$ , distinguishing real and generated samples. The discriminator aims to **maximize** cross-entropy.
- **Generator:** The generative model  $\mathbf{x} = \mathbf{G}_{\theta}(\mathbf{z})$ ,  $\mathbf{z} \sim p(\mathbf{z})$ , seeks to fool the discriminator. The generator aims to **minimize** cross-entropy.

## GAN Objective

$$\min_G \max_D \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x})) \right]$$

# Generative Adversarial Networks (GAN)

## Cross-Entropy for Discriminative Model

$$\max_{p(y|\mathbf{x})} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log p(y = 1|\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log p(y = 0|\mathbf{x}) \right]$$

- **Discriminator:** A classifier  $p_{\phi}(y = 1|\mathbf{x}) = D_{\phi}(\mathbf{x}) \in [0, 1]$ , distinguishing real and generated samples. The discriminator aims to **maximize** cross-entropy.
- **Generator:** The generative model  $\mathbf{x} = \mathbf{G}_{\theta}(\mathbf{z})$ ,  $\mathbf{z} \sim p(\mathbf{z})$ , seeks to fool the discriminator. The generator aims to **minimize** cross-entropy.

## GAN Objective

$$\min_G \max_D \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x})) \right]$$

$$\min_G \max_D \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(\mathbf{G}(\mathbf{z}))) \right]$$

**The last step is LOTUS** (Law of the Unconscious Statistician): use  $\mathbf{x} = \mathbf{G}(\mathbf{z})$  to rewrite the expectation over  $p_{\theta}(\mathbf{x})$  as an expectation over  $p(\mathbf{z})$ .

# GAN Optimality

## Theorem

The minimax game

$$\min_G \max_D \left[ \underbrace{\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(\mathbf{G}(\mathbf{z})))}_{V(G,D)} \right]$$

achieves its global optimum when  $p_{\text{data}}(\mathbf{x}) = p_{\theta}(\mathbf{x})$ , and  $D^*(\mathbf{x}) = 0.5$ .

# GAN Optimality

## Theorem

The minimax game

$$\min_G \max_D \underbrace{\left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(\mathbf{G}(\mathbf{z}))) \right]}_{V(G,D)}$$

achieves its global optimum when  $p_{\text{data}}(\mathbf{x}) = p_{\theta}(\mathbf{x})$ , and  $D^*(\mathbf{x}) = 0.5$ .

## Proof (Fixed $G$ )

$$V(G, D) = \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x}))$$

# GAN Optimality

## Theorem

The minimax game

$$\min_G \max_D \left[ \underbrace{\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(\mathbf{G}(\mathbf{z})))}_{V(G,D)} \right]$$

achieves its global optimum when  $p_{\text{data}}(\mathbf{x}) = p_{\theta}(\mathbf{x})$ , and  $D^*(\mathbf{x}) = 0.5$ .

## Proof (Fixed $G$ )

$$\begin{aligned} V(G, D) &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x})) \\ &= \int \underbrace{[p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_{\theta}(\mathbf{x}) \log(1 - D(\mathbf{x}))]}_{y(D)} d\mathbf{x} \end{aligned}$$

# GAN Optimality

## Theorem

The minimax game

$$\min_G \max_D \underbrace{\left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(\mathbf{G}(\mathbf{z}))) \right]}_{V(G,D)}$$

achieves its global optimum when  $p_{\text{data}}(\mathbf{x}) = p_{\theta}(\mathbf{x})$ , and  $D^*(\mathbf{x}) = 0.5$ .

## Proof (Fixed $G$ )

$$\begin{aligned} V(G, D) &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log(1 - D(\mathbf{x})) \\ &= \int \underbrace{[p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_{\theta}(\mathbf{x}) \log(1 - D(\mathbf{x}))]}_{y(D)} d\mathbf{x} \\ \frac{dy(D)}{dD} &= \frac{p_{\text{data}}(\mathbf{x})}{D(\mathbf{x})} - \frac{p_{\theta}(\mathbf{x})}{1 - D(\mathbf{x})} = 0 \quad \Rightarrow \quad D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \end{aligned}$$

# The Optimal Discriminator Responds to the Densities

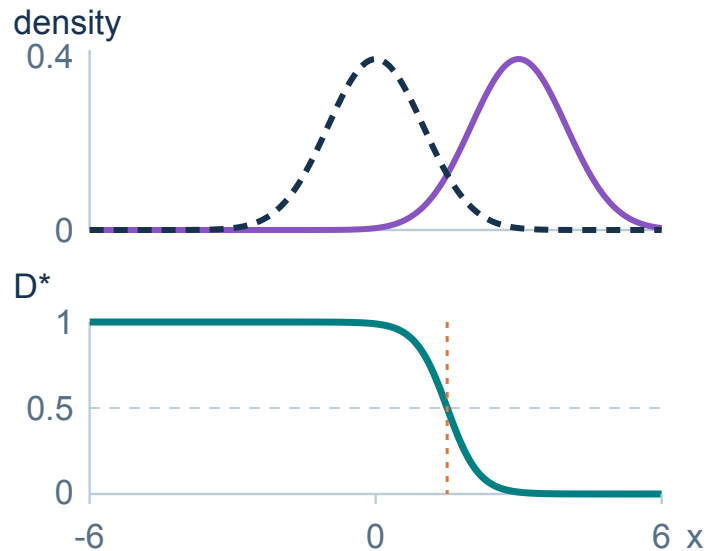
$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\theta}(x)}, \quad p_{\text{data}}(x) = \mathcal{N}(0, 1), \quad p_{\theta}(x) = \mathcal{N}(\mu, 1).$$

## Separated

 $\mu = 3$ 

$p_{\text{data}}(x) = \mathcal{N}(0, 1)$

$p_{\theta}(x) = \mathcal{N}(3, 1)$



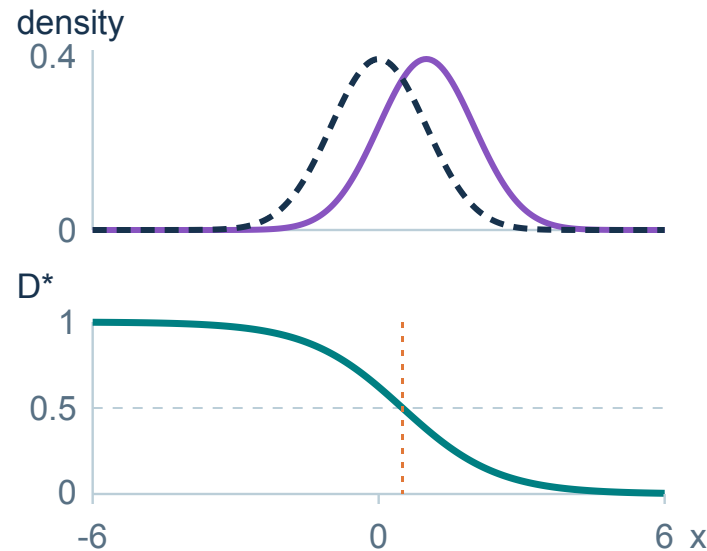
Equal densities at  $x = 1.5$ .

## Overlap

 $\mu = 1$ 

$p_{\text{data}}(x) = \mathcal{N}(0, 1)$

$p_{\theta}(x) = \mathcal{N}(1, 1)$



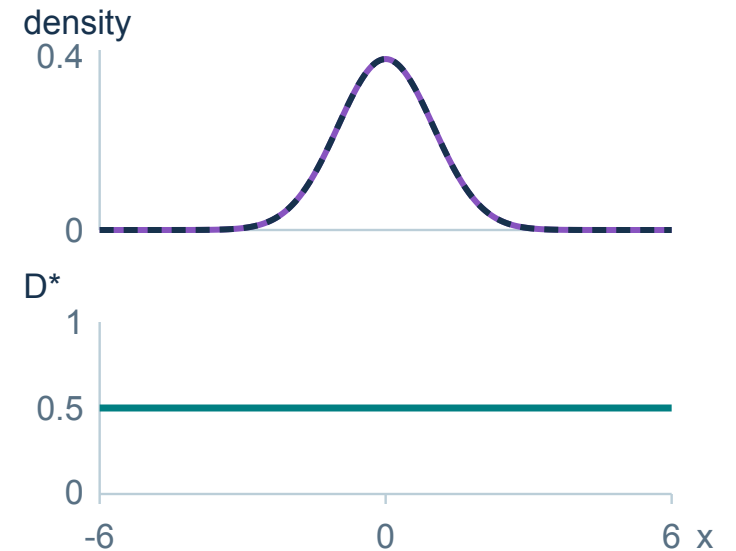
Equal densities at  $x = 0.5$ .

## Matched

 $\mu = 0$ 

$p_{\text{data}}(x) = \mathcal{N}(0, 1)$

$p_{\theta}(x) = \mathcal{N}(0, 1)$



Equal densities:  $D^* = 0.5$  everywhere.

Matching the generator to the data makes the optimal discriminator equal to 0.5 everywhere.

# GAN Optimality

## Proof Continued (Fixed $D = D^*$ )

$$V(G, D^*) = \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log \left( \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log \left( \frac{p_{\theta}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right)$$

# GAN Optimality

## Proof Continued (Fixed $D = D^*$ )

$$\begin{aligned} V(G, D^*) &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log \left( \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log \left( \frac{p_{\theta}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) \\ &= D_{\text{KL}} \left( p_{\text{data}}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) + D_{\text{KL}} \left( p_{\theta}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) - 2 \log 2 \end{aligned}$$

# GAN Optimality

## Proof Continued (Fixed $D = D^*$ )

$$\begin{aligned} V(G, D^*) &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log \left( \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log \left( \frac{p_{\theta}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) \\ &= D_{\text{KL}} \left( p_{\text{data}}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) + D_{\text{KL}} \left( p_{\theta}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) - 2 \log 2 \\ &= 2 D_{\text{JS}}(p_{\text{data}}(\mathbf{x}) \parallel p_{\theta}(\mathbf{x})) - 2 \log 2. \end{aligned}$$

# GAN Optimality

## Proof Continued (Fixed $D = D^*$ )

$$\begin{aligned}
 V(G, D^*) &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log \left( \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log \left( \frac{p_{\theta}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) \\
 &= D_{\text{KL}} \left( p_{\text{data}}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) + D_{\text{KL}} \left( p_{\theta}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) - 2 \log 2 \\
 &= 2 D_{\text{JS}}(p_{\text{data}}(\mathbf{x}) \parallel p_{\theta}(\mathbf{x})) - 2 \log 2.
 \end{aligned}$$

## Jensen-Shannon Divergence (Symmetric KL Divergence)

$$D_{\text{JS}}(p_{\text{data}}(\mathbf{x}) \parallel p_{\theta}(\mathbf{x})) = \frac{1}{2} \left[ D_{\text{KL}} \left( p_{\text{data}}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) + D_{\text{KL}} \left( p_{\theta}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) \right]$$

# GAN Optimality

## Proof Continued (Fixed $D = D^*$ )

$$\begin{aligned}
 V(G, D^*) &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log \left( \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) + \mathbb{E}_{p_{\theta}(\mathbf{x})} \log \left( \frac{p_{\theta}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})} \right) \\
 &= D_{\text{KL}} \left( p_{\text{data}}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) + D_{\text{KL}} \left( p_{\theta}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) - 2 \log 2 \\
 &= 2 D_{\text{JS}}(p_{\text{data}}(\mathbf{x}) \parallel p_{\theta}(\mathbf{x})) - 2 \log 2.
 \end{aligned}$$

## Jensen-Shannon Divergence (Symmetric KL Divergence)

$$D_{\text{JS}}(p_{\text{data}}(\mathbf{x}) \parallel p_{\theta}(\mathbf{x})) = \frac{1}{2} \left[ D_{\text{KL}} \left( p_{\text{data}}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) + D_{\text{KL}} \left( p_{\theta}(\mathbf{x}) \parallel \frac{p_{\text{data}}(\mathbf{x}) + p_{\theta}(\mathbf{x})}{2} \right) \right]$$

This can be regarded as a proper distance metric!

$$V(G^*, D^*) = -2 \log 2, \quad p_{\text{data}}(\mathbf{x}) = p_{\theta}(\mathbf{x}), \quad D^*(\mathbf{x}) = 0.5.$$

# GAN Optimality

## Theorem

The following minimax game

$$\min_G \max_D \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(\mathbf{G}(\mathbf{z}))) \right]$$

achieves its global optimum precisely when  $p_{\text{data}}(\mathbf{x}) = p_{\theta}(\mathbf{x})$ , and  $D^*(\mathbf{x}) = 0.5$ .

## Expectations

If the generator can express **any** function and the discriminator is **optimal** at every step, the generator **will converge** to the target distribution.

# GAN Optimality

## Theorem

The following minimax game

$$\min_G \max_D \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D(\mathbf{G}(\mathbf{z}))) \right]$$

achieves its global optimum precisely when  $p_{\text{data}}(\mathbf{x}) = p_{\theta}(\mathbf{x})$ , and  $D^*(\mathbf{x}) = 0.5$ .

## Expectations

If the generator can express **any** function and the discriminator is **optimal** at every step, the generator **will converge** to the target distribution.

## Reality

- Generator updates are performed in parameter space, and the discriminator is often imperfectly optimized.
- Generator and discriminator losses typically oscillate during GAN training.

# GAN Training

Assume both generator and discriminator are parametric models:  $D_\phi(\mathbf{x})$  and  $\mathbf{G}_\theta(\mathbf{z})$ .

## Objective

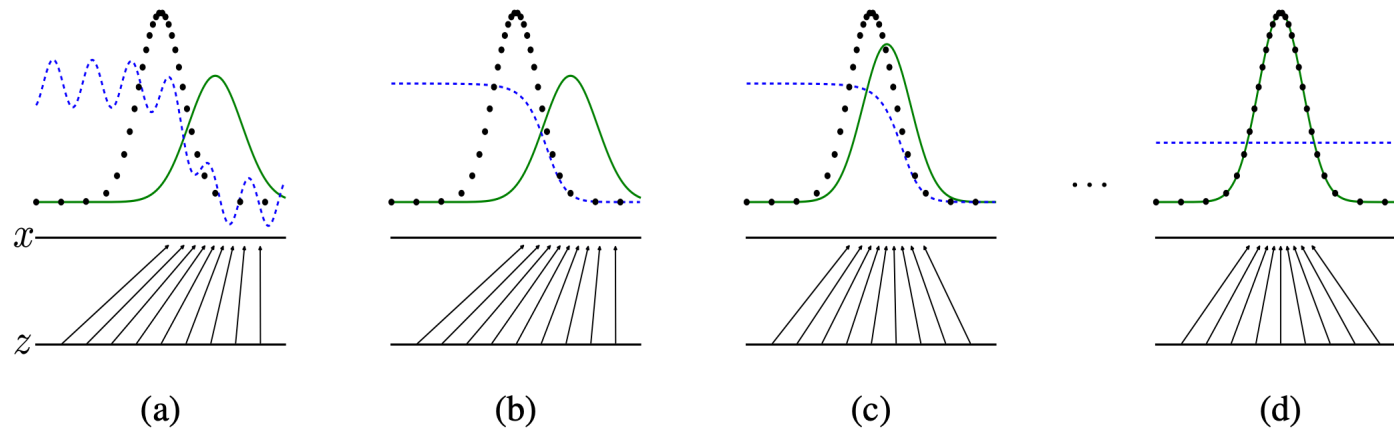
$$\min_{\theta} \max_{\phi} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D_\phi(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D_\phi(\mathbf{G}_\theta(\mathbf{z}))) \right]$$

# GAN Training

Assume both generator and discriminator are parametric models:  $D_\phi(\mathbf{x})$  and  $\mathbf{G}_\theta(\mathbf{z})$ .

## Objective

$$\min_{\theta} \max_{\phi} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D_\phi(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D_\phi(\mathbf{G}_\theta(\mathbf{z}))) \right]$$

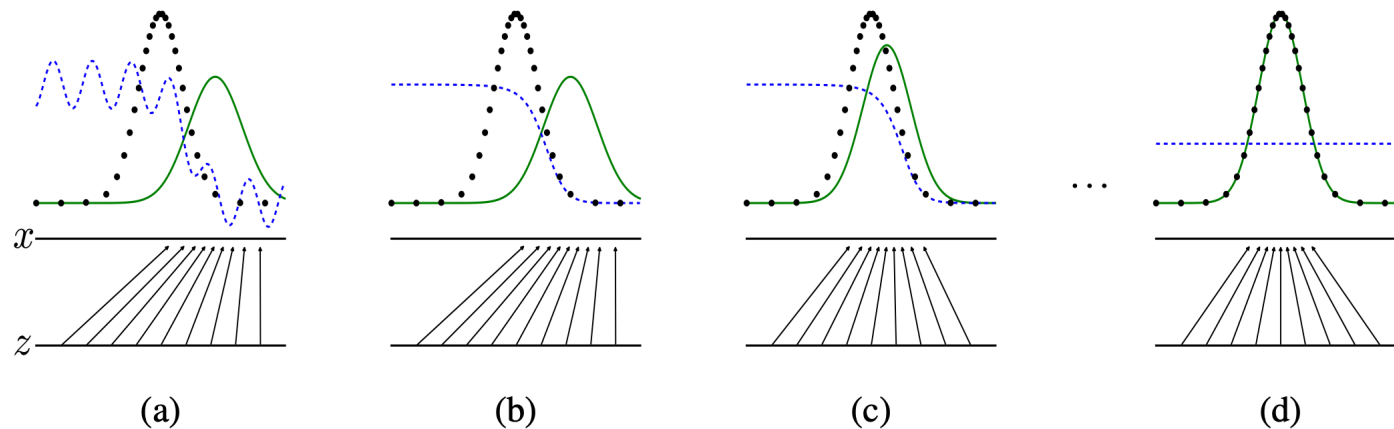


# GAN Training

Assume both generator and discriminator are parametric models:  $D_\phi(\mathbf{x})$  and  $\mathbf{G}_\theta(\mathbf{z})$ .

## Objective

$$\min_{\theta} \max_{\phi} \left[ \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log D_\phi(\mathbf{x}) + \mathbb{E}_{p(\mathbf{z})} \log(1 - D_\phi(\mathbf{G}_\theta(\mathbf{z}))) \right]$$



- $\mathbf{z} \sim p(\mathbf{z})$  is a latent variable.
- $p_\theta(\mathbf{x}|\mathbf{z}) = \delta(\mathbf{x} - \mathbf{G}_\theta(\mathbf{z}))$  serves as a deterministic decoder (like normalizing flows).
- There is no encoder present.

# Summary

- ELBO surgery gives insights into the prior's influence in VAEs; the optimal prior is the aggregated variational posterior.
- With a Gaussian decoder, averaging over distinct inputs that share a latent region can produce blurry outputs even when the prior matches the aggregated posterior.
- VQ-VAE learns discrete codes; a second stage fits their aggregated posterior with a generative prior.
- The straight-through gradient estimator allows gradients to pass as if quantization were an identity operation during backpropagation.
- Likelihood is not always a suitable metric for evaluating generative models; likelihood-free learning motivates the use of discriminative models to match distributions.
- GANs optimize the Jensen-Shannon divergence in theory; training alternates generator and discriminator updates to match the data distribution.